

Anexo II

Dpto. INFORMÁTICA – C.I.F.P. Cesar Manrique
MÓDULO PROYECTO
C.F.G.S. Desarrollo de Aplicaciones Web

Personal Finance Manager

Aplicación completa de gestión personal de finanzas.



Autor/es: Abraham Quintana Micó
Tutor: José David Díaz Díaz
Fecha: 10/05/2026

Sumario

Aplicación completa de gestión personal de finanzas.....	1
Introducción.....	6
Origen y Contextualización del Proyecto.....	6
Objetivo general del Proyecto Objetivos específicos.....	8
Objetivos específicos.....	8
Tareas.....	9
Tarea 1: Análisis de requisitos.....	9
Objetivo y metodología.....	9
Subtareas.....	9
Subtarea 1.1: Identificación de requisitos funcionales.....	9
Subtarea 1.2: Identificación de requisitos no funcionales.....	9
Subtarea 1. 3: Diseño de la arquitectura del sistema.....	10
Planificación.....	11
Tarea 2: Diseño del modelo entidad-relación.....	12
Objetivo y metodología.....	12
Subtareas.....	13
Subtarea 2.1: Análisis de requisitos técnicos y mapeos.....	13
Subtarea 2.2: Diseño conceptual por entidad.....	13
Subtarea 2.3: Definición de enums y reglas de negocio.....	13
Subtarea 2.4: Normalización y optimización.....	13
Subtarea 2.5: Diagramación y validación.....	13
Modelo ER.....	14
Planificación.....	14
Tarea 3: Inicialización del proyecto y análisis de dependencias.....	15
Objetivo y metodología.....	15
Subtareas.....	15
Subtarea 3.1 — Generación del proyecto con Spring Initializr.....	15
Subtarea 3.2 — Revisión y ajuste del pom.xml.....	15
Subtarea 3.3 — Estructura de paquetes y clases base.....	15
Subtarea 3.4 — Verificación de build y ejecución.....	15
Justificación de dependencias incluidas en el pom.xml.....	16
Planificación.....	18
Tarea 4: Implementación de modelos, repositorios, DTOs, mappers y seguridad; pruebas iniciales de autenticación.....	19
Objetivo y metodología.....	19
Subtareas.....	20
Subtarea 4.1 — Implementación de modelos y repositorios.....	20
Subtarea 4.2 — Definición de DTOs y mappers.....	20
Subtarea 4.3 — Configuración de seguridad y JWT.....	20
Subtarea 4.4 — Implementación de AuthController y servicios asociados.....	20
Subtarea 4.5 — Manejo de errores y pruebas de integración con Postman.....	21
Consideraciones técnicas y buenas prácticas.....	21
Checklist de pruebas con Postman (pasos y expectativas).....	21
Planificación.....	22
Tarea 5: Creación de controladores y servicios relacionados.....	23
Objetivo y metodología.....	23
Subtareas.....	23

Subtarea 5.1 — Implementación de UserController.....	23
Subtarea 5.2 — Implementación de CategoryController.....	24
Subtarea 5.3 — Implementación de TransactionController.....	24
Subtarea 5.4 — Implementación de AuthService.....	24
Subtarea 5.5 — Implementación de UserFinancialStateService.....	25
Subtarea 5.6 — Pruebas con Postman.....	25
Planificación.....	26
Tarea 6: Configuración de la base de datos PostgreSQL con Docker y adaptación del proyecto.....	27
Objetivo y metodología.....	27
Subtareass.....	27
Subtarea 6.1 — Creación del contenedor PostgreSQL con Docker Compose.....	27
Subtarea 6.2 — Modificación del fichero application.properties.....	28
Subtarea 6.3 — Verificación de persistencia y relaciones.....	28
Subtarea 6.4 — Verificación de encriptación de contraseñas.....	29
Planificación.....	29
Tarea 7: Creación del cliente React, dependencias iniciales y componentes base.....	30
Objetivo y metodología.....	30
Subtareass.....	30
Subtarea 7.1 — Inicialización del proyecto con create-react-app.....	30
Subtarea 7.2 — Instalación de dependencias necesarias.....	31
Subtarea 7.3 — Creación del logo y favicon.....	31
Subtarea 7.4 — Implementación del componente Header.....	31
Subtarea 7.5 — Implementación del componente Footer.....	32
Subtarea 7.6 — Implementación del componente Home.....	32
Subtarea 7.7 — Creación del RouterApp.....	32
Planificación.....	33
Tarea 8: Creación de utilidades, componentes de autenticación y servicios del cliente React.....	34
Objetivo y metodología.....	34
Subtareass.....	34
Subtarea 8.1 — Creación del fichero de constantes.....	34
Subtarea 8.2 — Implementación del componente Registry.....	35
Subtarea 8.3 — Implementación del componente Login.....	35
Subtarea 8.4 — Implementación del servicio authService.....	35
Subtarea 8.5 — Implementación del AuthContext.....	36
Subtarea 8.6 — Implementación del componente RequireAuth.....	36
Planificación.....	36
Tarea 9: Pruebas funcionales del cliente React (autenticación y navegación).....	37
Objetivo y metodología.....	37
Subtareass.....	37
Subtarea 9.1 — Verificación de las rutas generadas por authService.....	37
Subtarea 9.2 — Pruebas del componente Registry.....	38
Subtarea 9.3 — Pruebas del componente Login.....	38
Subtarea 9.4 — Verificación del comportamiento del Header.....	39
Subtarea 9.5 — Pruebas del AuthContext.....	40
Subtarea 9.6 — Pruebas del componente RequireAuth.....	40
Planificación.....	41
Tarea 10: Creación y pruebas del componente Profile y del servicio necesario userService.....	42
Objetivo y metodología.....	42
Subtareass.....	42
Subtarea 10.1 — Implementación del servicio userService.....	42
Subtarea 10.2 — Implementación del componente Profile.....	43

Subtarea 10.3 — Pruebas del servicio userService.....	43
Subtarea 10.4 — Pruebas del componente Profile.....	44
Subtarea 10.5 — Verificación de interacción con AuthContext y navegación.....	45
Planificación.....	45
Tarea 11: Creación y pruebas del componente Category y del servicio categoryService.....	46
Objetivo y metodología.....	46
Subtareas.....	46
Subtarea 11.1 — Implementación del servicio categoryService.....	46
Subtarea 11.2 — Implementación del componente Category.....	47
Subtarea 11.3 — Pruebas del servicio categoryService.....	47
Subtarea 11.4 — Pruebas del componente Category.....	48
Subtarea 11.5 — Verificación de interacción con AuthContext y navegación.....	49
Planificación.....	49
Tarea 12: Creación y pruebas del componente Transaction y del servicio transactionService....	50
Objetivo y metodología.....	50
Subtareas.....	50
Subtarea 12.1 — Implementación del servicio transactionService.....	50
Subtarea 12.2 — Implementación del componente Transaction.....	51
Subtarea 12.3 — Pruebas del servicio transactionService.....	52
Subtarea 12.4 — Pruebas del componente Transaction.....	53
Subtarea 12.5 — Verificación de interacción con AuthContext y navegación.....	54
Planificación.....	54
Tarea 13: Creación del componente Statistics, su servicio asociado y refactorización (extracción de subcomponentes).....	55
Objetivo y metodología.....	55
Subtareas.....	55
Subtarea 13.1 — Instalación de dependencias para gráficos.....	55
Subtarea 13.2 — Implementación del servicio statisticsService.....	56
Subtarea 13.3 — Implementación del componente Statistics.....	56
Subtarea 13.4 — Extracción de subcomponentes (refactorización).....	57
Subtarea 13.5 — Pruebas del servicio statisticsService.....	57
Subtarea 13.6 — Pruebas del componente Statistics.....	58
Planificación.....	59
Tarea 14: Creación del componente UserNav, inserción en pantallas protegidas y ajustes finales de estilos.....	60
Objetivo y metodología.....	60
Subtareas.....	60
Subtarea 14.1 — Creación del componente UserNav.....	60
Subtarea 14.2 — Inserción de UserNav en pantallas protegidas.....	61
Subtarea 14.3 — Revisión y ajustes del archivo styles.css.....	61
Subtarea 14.4 — Pruebas generales del cliente.....	62
Planificación.....	63
Tarea 15: Dockerización completa de la aplicación.....	64
Objetivo y metodología.....	64
Subtareas.....	64
Subtarea 15.1 — Dockerfile del servidor y pruebas de conexión a la BBDD.....	64
Subtarea 15.2 — Configuración de Nginx para la red interna.....	65
Subtarea 15.3 — Dockerfile del cliente con Nginx y pruebas de build.....	65
Subtarea 15.4 — Ajuste de rutas en el cliente y CORS en el servidor.....	65
Subtarea 15.5 — Creación del docker-compose.yml.....	66
Subtarea 15.6 — Pruebas completas con docker-compose.....	67

Planificación.....	67
Recursos humanos.....	68
Recursos materiales.....	68
Cronograma.....	69
Presupuesto.....	70
Política de seguimiento y evaluación.....	74
Futuras mejoras.....	75
Anexos.....	76
Bibliografía.....	77

INTRODUCCIÓN

El proyecto *Personal Finance Manager* consiste en el desarrollo de una aplicación web destinada a la gestión integral de finanzas personales. Su propósito es ofrecer una herramienta que permita registrar, organizar y analizar ingresos y gastos de manera automatizada, proporcionando al usuario una visión clara y actualizada de su situación económica. La aplicación incorpora funcionalidades orientadas al control financiero diario, la planificación mensual y el análisis histórico mediante representaciones gráficas.

El sistema permite realizar operaciones esenciales como el registro de movimientos económicos, la clasificación por categorías, el cálculo de balances y la consulta de métricas relevantes para la toma de decisiones. Asimismo, ofrece visualizaciones que facilitan la interpretación de la evolución financiera a lo largo del tiempo. La interfaz está diseñada para ser clara, accesible y adaptable a distintos dispositivos, garantizando una experiencia de uso fluida tanto en entornos de escritorio como móviles.

El proyecto se plantea como una solución técnica completa basada en tecnologías modernas de desarrollo web, integrando un backend construido con Spring Boot, un frontend desarrollado con React y una base de datos PostgreSQL, todo ello desplegado mediante contenedores Docker. Su finalidad es servir como demostración funcional de un sistema web realista, aplicable tanto a usuarios individuales como a pequeñas empresas que requieran un control financiero sencillo y visual.

ORIGEN Y CONTEXTUALIZACIÓN DEL PROYECTO

El proyecto surge de la necesidad de disponer de una herramienta digital que permita gestionar de forma centralizada y automatizada la información económica personal. En un contexto en el que la mayoría de usuarios realiza pagos electrónicos, suscripciones periódicas y operaciones financieras frecuentes, resulta habitual perder visibilidad sobre el balance real disponible, los gastos acumulados o la evolución del ahorro a lo largo del tiempo. Esta falta de control puede derivar en decisiones económicas poco informadas o en dificultades para planificar el presupuesto mensual.

La aplicación *Personal Finance Manager* se plantea como respuesta a esta problemática, proporcionando un sistema que facilita el registro estructurado de ingresos y gastos, la clasificación por categorías y el análisis de la situación financiera mediante indicadores y gráficos. Su diseño se orienta a usuarios particulares y pequeñas empresas que requieren una solución sencilla, accesible y visual para supervisar su actividad económica sin necesidad de conocimientos contables avanzados.

El proyecto se desarrolla dentro del ámbito del ciclo formativo de Desarrollo de Aplicaciones Web, integrando tecnologías actuales utilizadas en entornos profesionales. La elección de un backend basado en Spring Boot, un frontend construido con React y una base de datos PostgreSQL responde a la intención de reproducir un entorno realista de desarrollo web moderno. Asimismo, el uso de contenedores Docker permite simular un despliegue completo similar al que se emplea en entornos productivos.

La contextualización técnica y académica del proyecto lo convierte en un ejercicio práctico de integración de conocimientos, abarcando desde el diseño de la base de datos y la implementación de la lógica de negocio hasta la construcción de la interfaz de usuario y el despliegue de la aplicación. El resultado es una solución funcional que demuestra la capacidad de aplicar metodologías, herramientas y tecnologías propias del desarrollo web actual.

OBJETIVO GENERAL DEL PROYECTO OBJETIVOS ESPECÍFICOS

Desarrollar una aplicación web completa que permita a los usuarios gestionar sus finanzas personales de forma automatizada, estructurada y visual, facilitando el control de ingresos y gastos, el análisis del balance económico y la toma de decisiones financieras mediante indicadores y gráficos interactivos.

Objetivos específicos

- Implementar un sistema de registro y autenticación de usuarios que garantice el acceso seguro a la aplicación.
- Diseñar y desarrollar una base de datos estructurada que permita almacenar de forma eficiente transacciones, categorías y datos asociados al usuario.
- Crear un módulo de gestión de transacciones que permita registrar, consultar y eliminar ingresos y gastos.
- Incorporar un sistema de categorización que permita clasificar los movimientos económicos para facilitar su análisis.
- Calcular y mostrar indicadores financieros clave, como balance total, balance inicial del mes, gastos e ingresos acumulados y previsiones para el resto del mes.
- Implementar herramientas de análisis visual, incluyendo gráficos de evolución temporal y distribución por categorías.
- Desarrollar una interfaz web responsive, accesible desde distintos dispositivos y orientada a la facilidad de uso.
- Integrar todos los componentes mediante una API REST, garantizando la comunicación entre frontend, backend y base de datos.
- Desplegar la aplicación mediante contenedores Docker, simulando un entorno realista de producción.

TAREAS

Tarea 1: Análisis de requisitos

Objetivo y metodología

El objetivo de esta tarea es identificar, recopilar y definir de forma precisa las necesidades funcionales y no funcionales de la aplicación *Personal Finance Manager*. La tarea consiste en analizar qué funcionalidades debe ofrecer el sistema, cómo deben comportarse los distintos módulos y qué restricciones técnicas condicionan su desarrollo.

La metodología empleada se basa en un análisis funcional estructurado, que incluye la revisión de casos de uso habituales en aplicaciones de gestión financiera, la definición de flujos de usuario, la identificación de entidades principales y la delimitación del alcance del proyecto. Este proceso permite establecer una base sólida para el diseño posterior del modelo de datos, la arquitectura del backend y la estructura del frontend.

Subtareas

Subtarea 1.1: Identificación de requisitos funcionales

Consiste en determinar las acciones que el usuario podrá realizar dentro del sistema, tales como registro, autenticación, gestión de transacciones, consulta de balances y visualización de gráficos.

Subtarea 1.2: Identificación de requisitos no funcionales

Incluye aspectos como rendimiento, seguridad, usabilidad, adaptabilidad a distintos dispositivos y requisitos técnicos derivados del uso de Spring Boot, React y PostgreSQL.

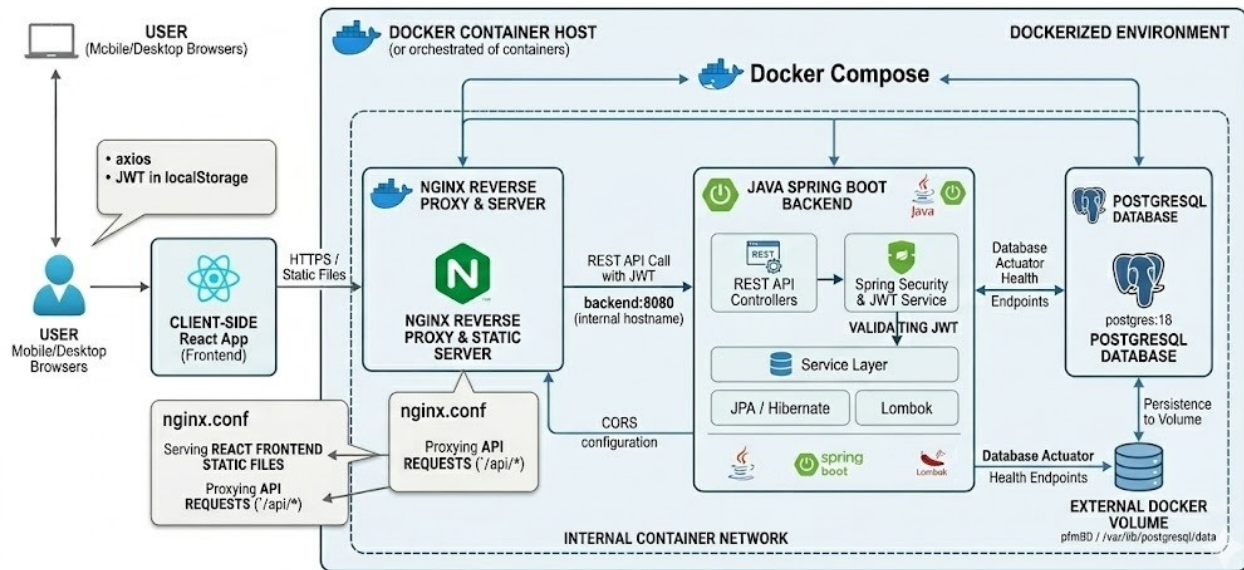
Subtarea 1. 3: Diseño de la arquitectura del sistema

Definición de la estructura técnica y los componentes principales del sistema para garantizar que los requisitos funcionales y no funcionales puedan implementarse de forma coherente y escalable. Esto incluye decidir tecnologías, límites de responsabilidad entre capas, patrones de comunicación y requisitos de despliegue iniciales.

Requisitos definidos:

- **Backend Spring Boot (Java 21 “LTS”):** Estructura interna del servidor de aplicaciones, incluyendo el controlador REST API, el servicio de seguridad que valida los tokens JWT (almacenados en el cliente tras el registro/login), la capa de servicio y el mapeo JPA/Hibernate.
- **Base de Datos PostgreSQL Persistente:** Contenedor de la base de datos PostgreSQL, utilizando un **Volumen Docker Externo** en el host para garantizar la persistencia de los datos, incluso tras reinicios de los contenedores.
- **Frontend React (Node 24):** Produciendo renderización dinámica y gestión del estado del cliente. Se comunica de forma asíncrona mediante **Axios** con el backend para procesar datos financieros y gestionar la autenticación por **JWT**. Su arquitectura basada en componentes garantiza una navegación fluida (SPA) y una experiencia de usuario reactiva y altamente optimizada.
- **Proxy inverso (Nginx última versión estable):** Todo el contenido estático se sirve de forma eficiente a través de **Nginx**, que actúa como servidor web y punto de entrada al ecosistema Dockerizado.
- **Entorno Docker Orquestado:** Docker Compose coordina los tres contenedores principales (frontend, backend, postgres) dentro de una red interna protegida, creando un entorno de producción realista.

PROJECT ARCHITECTURE - PERSONAL FINANCE MANAGER



Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 1: Análisis de requisitos	10 h	19/03/2026	22/03/2026
Subtarea 1.1: Identificación de requisitos funcionales	4 h	19/03/2026	19/03/2026
Subtarea 1.2: Definición de requisitos no funcionales.	4 h	20/03/2026	21/03/2026
Subtarea 1.3: Diseño inicial de la arquitectura del sistema	2 h	21/03/2026	22/03/2026

Tarea 2: Diseño del modelo entidad-relación

Objetivo y metodología

El objetivo es diseñar el modelo entidad-relación (ER) que represente de forma normalizada las entidades y relaciones necesarias para soportar las funcionalidades del sistema: gestión de usuarios, categorías y transacciones (incluyendo tipos y recurrencias). El modelo debe garantizar integridad referencial, facilitar consultas eficientes para los indicadores financieros y soportar la lógica de transacciones periódicas (mensuales/anuales).

La metodología a emplear es un enfoque de diseño conceptual seguido de normalización y validación:

- **Análisis conceptual:** identificación de entidades, atributos y relaciones a partir de los requisitos funcionales.
- **Definición de claves y restricciones:** establecimiento de claves primarias, claves foráneas, unicidad y restricciones de nulabilidad.
- **Modelado de enumeraciones:** definición de tipos enumerados para TransactionType y RecurrenceType.
- **Validación:** revisión del modelo frente a casos de uso (registro de transacciones pasadas y futuras; generación automática de recurrencias).
- **Entrega gráfica:** diagrama ER en formato PNG que refleje entidades, atributos, PK/FK y cardinalidades.

Subtareas

Subtarea 2.1: Análisis de requisitos técnicos y mapeos

Revisión de los requisitos funcionales y no funcionales que afectan al modelo (consultas frecuentes, índices necesarios, integridad y restricciones).

Subtarea 2.2: Diseño conceptual por entidad

Definición de atributos y relaciones para cada entidad: Usuario, Categoría, Transacción.

Subtarea 2.3: Definición de enums y reglas de negocio

Creación de TransactionType (INCOME, EXPENSE) y RecurrenceType (NO, MONTHLY, YEARLY) y especificación de su almacenamiento (STRING).

Subtarea 2.4: Normalización y optimización

Aplicar normalización y proponer índices para consultas críticas (por fecha, por usuario, por categoría).

Subtarea 2.5: Diagramación y validación

Generación del diagrama ER y revisión con casos de uso (incluyendo transacciones periódicas y replicación automática).

Consideraciones funcionales específicas

- Las transacciones pueden ser pasadas o futuras.
- Se debe soportar la creación de transacciones con recurrencia mensual y anual.
- Un servicio replicará automáticamente las transacciones recurrentes al pasar la fecha de la transacción, creando una nueva instancia con la nueva fecha y marcando la anterior como no recurrente (detalles de implementación del servicio y repositorios se definirán en la fase de desarrollo).
- User.email y Category.name deben ser únicos por diseño.
- Category está vinculada a User (cada usuario tiene su propio conjunto de categorías).
- Transaction referencia obligatoriamente a User y a Category.
- Data Definition Language no es necesario ya que JPA junto con la propiedad (spring.jpa.hibernate.ddl-auto=update) genera automáticamente las tablas, relaciones y demás.

Modelo ER

A continuación se incluye el diagrama entidad-relación que representa el modelo resultante (entidades, atributos, relaciones).

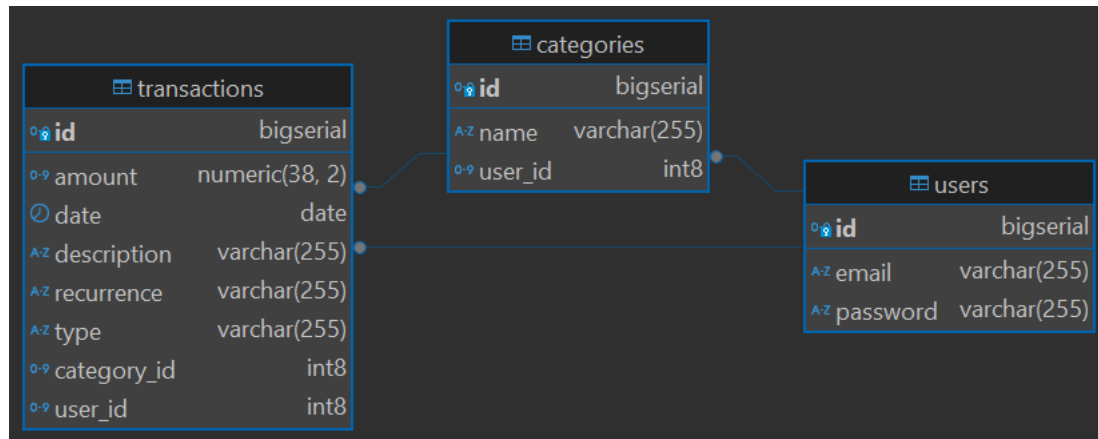


Diagrama generado con Dbeaver

Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 2: Diseño del modelo Entidad Relación	4,5 h	22/03/2026	22/03/2026
Subtarea 2.1 — Análisis requisitos técnicos	1 h	22/03/2026	22/03/2026
Subtarea 2.2 — Diseño conceptual por entidad	1 h	22/03/2026	22/03/2026
Subtarea 2.3 — Definición de enums y reglas	0,5 h	22/03/2026	22/03/2026
Subtarea 2.4 — Normalización y optimización	1 h	22/03/2026	22/03/2026
Subtarea 2.5 — Diagramación y validación	1 h	22/03/2026	22/03/2026

Tarea 3: Inicialización del proyecto y análisis de dependencias

Objetivo y metodología

Objetivo. Inicializar el proyecto backend con Spring Boot, configurar el pom.xml y los plugins necesarios, establecer la estructura de paquetes y preparar el entorno para el desarrollo (perfil de desarrollo con H2 y perfil de producción con PostgreSQL). Además, justificar y documentar las dependencias seleccionadas para que su uso quede claramente motivado en la memoria técnica.

Metodología. Se parte de Spring Initializr para generar la base del proyecto y se procede a revisar y ajustar el pom.xml aportado. El trabajo incluye: verificación de la versión de Java, agrupación de starters de Spring Boot, explicación de dependencias no estándar (bibliotecas JWT, drivers de BBDD, Lombok, JPA, ...), configuración de plugins de compilación y anotación, y creación de configuraciones para H2(bbdd en memoria para desarrollo) y PostgreSQL(para persistencia real en producción). Finalmente se realiza una compilación y ejecución inicial para validar la configuración.

Subtareas

Subtarea 3.1 — Generación del proyecto con Spring Initializr

Crear el proyecto base con groupId, artifactId, java.version y los starters mínimos; descargar y abrir en el IDE IntelliJ.

Subtarea 3.2 — Revisión y ajuste del pom.xml

Incorporar y justificar dependencias, configurar maven-compiler-plugin y spring-boot-maven-plugin, y añadir el annotationProcessorPaths para Lombok.

Subtarea 3.3 — Estructura de paquetes y clases base

Crear paquetes (controller, service, repository, model, config, dto, security, exceptions, mapper) y clases iniciales (clase Application, configuración de seguridad “JWT, FilterChain, Cors”, propiedades de la aplicación).

Subtarea 3.4 — Verificación de build y ejecución

Ejecutar mvn clean package y mvn spring-boot:run ; comprobar endpoints básicos y la consola H2.

Justificación de dependencias incluidas en el pom.xml

Dependencias agrupadas (starters de Spring Boot)

- `spring-boot-starter-data-jpa`: Proporciona integración con JPA/Hibernate, gestión de transacciones y repositorios Spring Data; facilita el mapeo objeto-relacional y operaciones CRUD.
- `spring-boot-starter-security`: Añade la infraestructura de seguridad (filtros, configuración de autenticación/autorización) sobre la que se implementará la autenticación JWT.
- `spring-boot-starter-validation`: Incluye soporte para Bean Validation (JSR-380) y anotaciones como `@Valid`, `@NotNull`, `@Size` para validar DTOs y entidades.
- `spring-boot-starter-web`: Proporciona Spring MVC, Tomcat embebido y utilidades para exponer API REST; incluye Jackson para serialización JSON.
- `spring-boot-starter-actuator`: Añade endpoints de monitorización y salud (health, metrics) útiles durante desarrollo y especialmente despliegue con Docker.

Nota: los starters de Spring Boot agrupan dependencias transversales necesarias (logging, Jackson, Tomcat, etc.), por eso se pueden considerar dependencias “por defecto” en proyectos web con Spring Boot.

Dependencias específicas no-starter

- **`io.jsonwebtoken:jjwt-api / jjwt-impl / jjwt-jackson` (0.11.5)**: Biblioteca para creación y verificación de JWT.
 - `jjwt-api`: API pública.
 - `jjwt-impl` y `jjwt-jackson` en runtime scope: implementación y soporte de serialización con Jackson; se marcan en runtime porque no son necesarias en tiempo de compilación para compilar contra la API.

Justificación: se usará JWT para la autenticación stateless en la API REST; JJWT es una opción segura, actual y sencilla de integrar con Spring Security.

- **`com.h2database:h2` (scope runtime)**: Base de datos en memoria/archivo para desarrollo y pruebas locales.

Justificación: permite arranque rápido sin dependencias externas y facilita pruebas unitarias/integración en entorno local.

- **org.postgresql:postgresql** (scope runtime): Driver JDBC para PostgreSQL.

Justificación: base de datos objetivo en producción; se mantiene en runtime porque no es necesaria para compilar.

- **org.projectlombok:lombok**: Reduce boilerplate (getters/setters, constructores) mediante anotaciones.

Justificación: mejora la legibilidad del código y acelera el desarrollo.

- **spring-boot-starter-test** (scope test): Conjunto de utilidades para pruebas (JUnit, Mockito, Spring Test).

Justificación: imprescindible para escribir pruebas unitarias e integración. (debido a la escasez de tiempo los test finalmente quedan pendientes para desarrollar más adelante, pero la dependencia es integrada en los pasos iniciales).

Plugins y configuración de build

- **maven-compiler-plugin**: Configurado para Java 21 y con `annotationProcessorPaths` apuntando a Lombok para que las anotaciones se procesen correctamente durante la compilación.

Justificación: asegura que el código se compile con la versión de Java objetivo y que Lombok funcione en el proceso de compilación.

- **spring-boot-maven-plugin**: Permite empaquetar la aplicación como jar ejecutable y ejecutar `spring-boot:run`.

Justificación: facilita el ciclo de desarrollo y el despliegue.

Propiedad

java.version = 21

Justificación: se utiliza una versión LTS(Long Time Supported) de Java para el proyecto, elegir Java 21 permite aprovechar mejoras de rendimiento y características modernas del lenguaje, siempre que el entorno de despliegue lo soporte.

Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 3: Inicialización del proyecto	4,5 h	23/03/2026	23/03/2026
Subtarea 3.1 — Generación con Spring Initializr	1 h	23/03/2026	23/03/2026
Subtarea 3.2 — Revisión y ajuste del pom.xml	0,5 h	23/03/2026	23/03/2026
Subtarea 3.3 — Estructura de paquetes y clases	1.5 h	23/03/2026	23/03/2026
Subtarea 3.4 — Verificación build y ejecución	1.5 h	23/03/2026	23/03/2026

Tarea 4: Implementación de modelos, repositorios, DTOs, mappers y seguridad; pruebas iniciales de autenticación

Objetivo y metodología

Objetivo. Implementar las capas de persistencia y transporte de datos (entidades JPA, repositorios Spring Data, DTOs y mappers), configurar la seguridad basada en JWT (filtros, servicio y configuración de Spring Security), exponer el controlador de autenticación y verificar mediante pruebas con Postman el correcto registro y login de usuarios. Garantizar que las contraseñas se almacenan cifradas y que los errores de autenticación devuelven códigos HTTP adecuados.

Metodología. Se sigue un enfoque incremental y modular:

- **Implementación de entidades y repositorios:** crear las entidades JPA, interfaces Repository con consultas necesarias (p. ej. findByEmail).
- **DTOs y mappers:** definir DTOs para entrada/salida en la API (AuthRequest/AuthResponse, UserDTO, CategoryDTO, TransactionDTO, ...) y mappers sencillos (mapeo manual) para desacoplar la capa de persistencia de la API.
- **Seguridad:** integrar PasswordEncoder (BCrypt), JwtService para generación/validación de tokens, JwtFilter para extraer y validar tokens en cada petición y SecurityConfig para definir reglas de acceso.
- **Controlador de autenticación:** exponer endpoints /api/auth/register y /api/auth/login que devuelvan AuthResponse con token JWT. Manejar errores con excepciones mapeadas a códigos HTTP (404 para usuario no encontrado, 401 para credenciales inválidas).
- **Pruebas:** realizar pruebas funcionales con Postman: registro, login exitoso (comprobación de token) y login de usuario inexistente (comprobación de 404).
- **Verificación de seguridad de datos:** comprobar en la BBDD que el campo password contiene el hash (BCrypt) y no la contraseña en claro.

Subtareas

Subtarea 4.1 — Implementación de modelos y repositorios

Descripción: Asegurar que las entidades User, Category y Transaction están completas (anotaciones JPA, relaciones, constraints). Crear interfaces UserRepository, CategoryRepository, TransactionRepository extendiendo JpaRepository. Añadir método Optional<User> findByEmail(String email) en UserRepository.

Resultado esperado: Repositorios listos para inyección y uso en servicios y controladores.

Subtarea 4.2 — Definición de DTOs y mappers

Descripción: Crear DTOs para las operaciones públicas: AuthRequest, AuthResponse, UserDTO, CategoryDTO, TransactionDTO. Implementar mappers sencillos (métodos estáticos o clases utilitarias) que conviertan entre entidad ↔ DTO. Evitar exponer campos sensibles (p. ej. password) en DTOs de salida.

Resultado esperado: Contratos de API claros y desacoplados de las entidades.

Subtarea 4.3 — Configuración de seguridad y JWT

Descripción: Integrar PasswordEncoder (BCrypt), JwtService (generación/validación), JwtFilter (extracción del header Authorization y autenticación en SecurityContext), y SecurityConfig (reglas de acceso, CORS, deshabilitar CSRF para API). Asegurar que rutas públicas incluyen /api/auth/**, /h2-console/** y /actuator/**.

Puntos clave:

Cifrado de contraseña: usar passwordEncoder.encode(plain) al registrar.

Validación: en login, comparar con passwordEncoder.matches(raw, encoded).

Manejo de excepciones: mapear excepciones de negocio a respuestas HTTP (ver Subtarea 4.5).

Resultado esperado: flujo de autenticación JWT funcional y seguro.

Subtarea 4.4 — Implementación de AuthController y servicios asociados

Descripción: Implementar AuthController con endpoints POST /api/auth/register y POST /api/auth/login. Implementar (o delegar a) un AuthService que encapsule la lógica

de registro y autenticación (creación de usuario, hashing de contraseña, generación de token).

Comportamiento esperado:

Register: guarda usuario con contraseña codificada y devuelve `AuthResponse{token}`.

Login: busca usuario por email; si no existe devolver 404; si existe y contraseña válida devolver token; si contraseña inválida devolver 401.

Subtarea 4.5 — Manejo de errores y pruebas de integración con Postman

Descripción: Implementar manejo de errores consistente.

Pruebas Postman: crear colección con requests para register y login, validar token en respuesta y probar login con usuario inexistente.

Resultado esperado: pruebas reproducibles que confirmen el comportamiento esperado.

Consideraciones técnicas y buenas prácticas

- **Almacenamiento de contraseñas:** usar `BCryptPasswordEncoder` (definido en `SecurityConfig`) para que las contraseñas se almacenen como hashes irreversibles; comprobar en la BBDD que el campo `password` contiene un hash.
- **Generación de JWT:** `JwtService` debe usar una clave segura. La clave se genera en memoria con `Keys.secretKeyFor(...)`;
- **Validación de tokens:** `JwtFilter` valida el token y establece la autenticación en el `SecurityContext`. En la versión inicial se usa el email sin roles.
- **DTOs y seguridad:** nunca devolver el campo `password` en respuestas; usar DTOs de salida que excluyan datos sensibles.

Checklist de pruebas con Postman (pasos y expectativas)

1. Registro de usuario

- **Request:** POST `/api/auth/register`
- **Payload JSON:** `{"email":"user@example.com","password":"Secret123!"}`
- **Esperado:** 200 OK (o 201 Created) con body `{"token":"<JWT>"}`.

- **Verificación adicional:** comprobar en la BBDD que users.password contiene hash (no la contraseña en claro).

2. Login usuario existente

- **Request:** POST /api/auth/login
- **Payload JSON:** {"email":"user@example.com","password":"Secret123!"}
- **Esperado:** 200 OK con body {"token":"<JWT>"}
- **Verificación adicional:** decodificar el JWT (payload) para comprobar sub o email coincide.

3. Login usuario inexistente

- **Request:** POST /api/auth/login
- **Payload JSON:** {"email":"noexiste@example.com","password":"x"}
- **Esperado:** 404 Not Found con mensaje claro (User not found).

4. Login con contraseña incorrecta

- **Request:** POST /api/auth/login con email existente y contraseña errónea.
- **Esperado:** 401 Unauthorized con mensaje Invalid credentials.

Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 4: Implementación modelos y seguridad	20 h	24/03/2026	27/03/2026
Subtarea 4.1 — Modelos y repositorios	6 h	24/03/2026	24/03/2026
Subtarea 4.2 — DTOs y mappers	4 h	24/03/2026	25/03/2026
Subtarea 4.3 — Configuración de seguridad y JWT	4 h	25/03/2026	26/03/2026
Subtarea 4.4 — AuthController y AuthService	3 h	26/03/2026	26/03/2026
Subtarea 4.5 — Pruebas Postman y manejo errores	3 h	27/03/2026	27/03/2026

Tarea 5: Creación de controladores y servicios relacionados

Objetivo y metodología

Objetivo. Implementar los controladores REST del sistema (UserController, CategoryController, TransactionController) junto con los servicios necesarios (AuthService y UserFinancialStateService) para gestionar la información del usuario autenticado, las categorías, las transacciones y el estado financiero mensual. Todos los endpoints deben operar exclusivamente sobre los datos pertenecientes al usuario autenticado mediante JWT.

Metodología. La tarea se desarrolla siguiendo un enfoque estructurado:

- **Controladores REST:** implementación de los controladores que exponen operaciones CRUD para categorías y transacciones, así como endpoints para obtener el perfil del usuario y su estado financiero.
- **Servicios asociados:** implementación de AuthService para obtener el usuario autenticado desde el contexto de seguridad, y UserFinancialStateService para calcular el estado financiero completo del usuario.
- **Validación de propiedad:** en cada operación se comprueba que los recursos (categorías o transacciones) pertenecen al usuario autenticado.
- **DTOs y mapeo:** uso de DTOs y mappers existentes para garantizar que las respuestas no exponen información sensible.
- **Pruebas con Postman:** verificación manual del funcionamiento de los endpoints, incluyendo autenticación previa y uso del token en cabeceras.

Subtareas

Subtarea 5.1 — Implementación de UserController

Descripción: Implementar el controlador encargado de exponer información del usuario autenticado. Incluye:

- GET /api/user/me: devuelve un UserDto con la información básica del usuario.
- GET /api/user/state: devuelve un UserFinancialStateDto generado por UserFinancialStateService.

Resultado esperado: El usuario autenticado puede consultar su perfil y su estado financiero mensual.

Subtarea 5.2 — Implementación de CategoryController

Descripción: Implementar el controlador REST para gestionar categorías propiedad del usuario autenticado. Incluye:

- GET /api/categories: listado de categorías del usuario.
- POST /api/categories: creación de una nueva categoría asociada al usuario.
- PUT /api/categories/{id}: actualización de una categoría si pertenece al usuario.
- DELETE /api/categories/{id}: eliminación de una categoría si pertenece al usuario.

Resultado esperado: El usuario puede gestionar sus categorías de forma segura y aislada del resto de usuarios.

Subtarea 5.3 — Implementación de TransactionController

Descripción: Implementar el controlador REST para gestionar transacciones del usuario autenticado. Incluye:

- GET /api/transactions: listado de transacciones del usuario.
- POST /api/transactions: creación de una transacción asociada al usuario y a una categoría que también le pertenezca.
- DELETE /api/transactions/{id}: eliminación de una transacción si pertenece al usuario.

Resultado esperado: El usuario puede consultar, crear y eliminar sus transacciones de forma segura. Las transacciones no se podrán editar una vez creadas, si el usuario desea corregir una transacción, deberá eliminar la actual y crear una nueva.

Subtarea 5.4 — Implementación de AuthService

Descripción: Implementar el servicio encargado de obtener el usuario autenticado a partir del email incluido en el JWT. Utiliza:

- SecurityContextHolder.getContext().getAuthentication().getName()
- UserRepository.findByEmail(...)

Resultado esperado: Cualquier controlador puede obtener el usuario autenticado sin duplicar lógica.

Subtarea 5.5 — Implementación de UserFinancialStateService

Descripción: Implementar el servicio responsable de calcular el estado financiero del usuario. Incluye:

- Generación de transacciones recurrentes cuando corresponda.
- Obtención de transacciones del mes actual y siguiente.
- Filtrado de transacciones pasadas y futuras.
- Cálculo de ingresos, gastos, balance actual, balance inicial del mes, disponible diario y disponible hoy.

Resultado esperado: El sistema devuelve un UserFinancialStateDto completo y coherente con los datos del usuario.

Subtarea 5.6 — Pruebas con Postman

Descripción: Realizar pruebas manuales para verificar el correcto funcionamiento de los controladores. Las pruebas requieren **hacer login primero** y **usar el token JWT** en la cabecera:

Authorization: Bearer <token>

Ejemplos de pruebas

1. Obtener perfil del usuario

Request: GET /api/user/me **Headers:** Authorization: Bearer <token> **Esperado:** 200 OK con UserDto.

2. Crear categoría

Request: POST /api/categories **Headers:** Authorization: Bearer <token> **Body JSON:**

json

```
{ "name": "Comida" }
```

Esperado: 200 OK con CategoryDto.

3. Crear transacción

Request: POST /api/transactions **Headers:** Authorization: Bearer <token> **Body JSON:**

json

```
{  
  "amount": 50.00,  
  "date": "2026-03-15",  
  "description": "Compra supermercado",  
  "type": "EXPENSE",  
  "categoryId": 1,  
  "recurrence": "NO"  
}
```

Esperado: 200 OK con TransactionDto.

4. Obtener estado financiero

Request: GET /api/user/state **Headers:** Authorization: Bearer <token> **Esperado:** 200 OK con UserFinancialStateDto.

Resultado esperado: Todas las operaciones funcionan correctamente cuando el token es válido y solo afectan a los recursos del usuario autenticado.

Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 5: Controladores y servicios	23 h	28/03/2026	30/03/2026
Subtarea 5.1 — UserController	3 h	28/03/2026	28/03/2026
Subtarea 5.2 — CategoryController	4 h	28/03/2026	29/03/2026
Subtarea 5.3 — TransactionController	4 h	29/03/2026	29/03/2026
Subtarea 5.4 — AuthService	2 h	29/03/2026	29/03/2026
Subtarea 5.5 — UserFinancialStateService	8 h	30/03/2026	30/03/2026
Subtarea 5.6 — Pruebas Postman	2 h	31/03/2026	31/03/2026

Tarea 6: Configuración de la base de datos PostgreSQL con Docker y adaptación del proyecto

Objetivo y metodología

Objetivo. Configurar una base de datos PostgreSQL mediante Docker, sustituir la base de datos H2 utilizada durante el desarrollo inicial y adaptar la aplicación para que utilice PostgreSQL como origen de datos principal. Verificar que la persistencia funciona correctamente, incluyendo la integridad de las relaciones entre tablas y la correcta encriptación de contraseñas.

Metodología. La tarea se desarrolla en tres fases:

- **Configuración de la base de datos con Docker:** creación de un docker-compose.yml temporal para levantar un contenedor PostgreSQL con credenciales, volumen persistente y puerto expuesto.
- **Adaptación del proyecto:** modificación del fichero application.properties para apuntar a PostgreSQL en lugar de H2, ajustando URL, usuario, contraseña y dialecto.
- **Verificación:** ejecución de pruebas manuales para confirmar que los datos se guardan correctamente, que las relaciones entre entidades se mantienen y que las contraseñas se almacenan en formato encriptado (BCrypt).

Subtareas

Subtarea 6.1 — Creación del contenedor PostgreSQL con Docker Compose

Descripción: Crear un fichero docker-compose.yml temporal con la configuración necesaria para levantar un contenedor PostgreSQL. Este fichero servirá como base para el fichero de configuración Docker-compose final.

El fichero de configuración resultante es:

```
services:
  postgres:
    image: postgres:18
    container_name: postgres_for_personal_financial_manager
    environment:
      POSTGRES_PASSWORD: PFMAAdmin
```

POSTGRES_USER: postgres

POSTGRES_DB: pfm

ports:

- "5432:5432"

volumes:

- pfmBD:/var/lib/postgresql

volumes:

pfmBD:

El volumen pfmBD garantiza la persistencia de datos entre reinicios del contenedor.

Resultado esperado: El contenedor PostgreSQL se inicia correctamente y queda accesible en localhost:5432.

Subtarea 6.2 — Modificación del fichero application.properties

Descripción: Actualizar la configuración de Spring Boot para que utilice PostgreSQL como base de datos principal. Se sustituyen las propiedades de H2 por las correspondientes a PostgreSQL, incluyendo:

- URL JDBC
- Usuario y contraseña
- Driver
- Dialecto de Hibernate
- Configuración de ddl-auto según necesidades del proyecto

Resultado esperado: La aplicación arranca correctamente conectándose a PostgreSQL.

Subtarea 6.3 — Verificación de persistencia y relaciones

Descripción: Realizar pruebas manuales aprovechando las descritas en tareas anteriores para confirmar que:

- Los datos creados desde la aplicación se almacenan en PostgreSQL.
- Las relaciones entre tablas (users, categories, transactions) se mantienen correctamente.
- Las operaciones CRUD funcionan igual que antes del cambio de base de datos.

- Se siguen las reglas de unicidad, generación de identies, etc igual que pasaba cuando se usaba H2

Resultado esperado: Los datos persisten correctamente en el volumen Docker y las relaciones entre entidades se reflejan en la base de datos. Tras reiniciar el contenedor de Docker que contiene la imagen de Postgres, los datos siguen en la bbdd

Subtarea 6.4 — Verificación de encriptación de contraseñas

Descripción: Comprobar que las contraseñas almacenadas en la tabla users están en formato encriptado (BCrypt), tal como se genera en el registro de usuario mediante `passwordEncoder.encode(...)`.

Resultado esperado: El campo password contiene hashes BCrypt y no contraseñas en texto plano.

Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 6: Configuración PostgreSQL y adaptación	7 h	01/04/2026	01/04/2026
Subtarea 6.1 — Docker Compose PostgreSQL	2 h	01/04/2026	01/04/2026
Subtarea 6.2 — Modificación del fichero application.properties	2 h	01/04/2026	01/04/2026
Subtarea 6.3 — Verificación de persistencia	2 h	01/04/2026	01/04/2026
Subtarea 6.4 — Verificación de encriptación	1 h	01/04/2026	01/04/2026

Tarea 7: Creación del cliente React, dependencias iniciales y componentes base

Objetivo y metodología

Objetivo. Inicializar el cliente web del proyecto utilizando *create-react-app*, definir las dependencias necesarias para el funcionamiento de la aplicación, crear el logo que se utilizará en la cabecera y como favicon, e implementar los componentes base que estarán presentes en todas las páginas: Header, Footer, Home y el enrutador principal RouterApp.

Metodología. La tarea se desarrolla siguiendo un flujo ordenado:

- **Inicialización del proyecto React:** creación del cliente mediante create-react-app, estructura inicial de carpetas y configuración base.
- **Instalación de dependencias:** incorporación de librerías necesarias para el enrutado, contexto de autenticación y estilos.
- **Creación del logo:** diseño y colocación del archivo logo.png en la carpeta /assets, así como su uso como favicon.
- **Implementación de componentes base:** creación de Header, Footer y Home que aparecerán en toda la aplicación.
- **Configuración del enrutador:** creación del componente RouterApp que incluye cabecera, pie de página y las rutas principales del proyecto.

Subtareas

Subtarea 7.1 — Inicialización del proyecto con create-react-app

Descripción: Crear el cliente ejecutando:

```
npm create-react-app personal-finance-manager-frontend
```

Organizar la estructura inicial del proyecto, incluyendo las carpetas:

- components
- context
- assets

- router
- services
- styles
- utils

Resultado esperado: Proyecto React funcional con estructura base lista para añadir componentes.

Subtarea 7.2 — Instalación de dependencias necesarias

Descripción: Instalar las dependencias iniciales utilizadas en el proyecto:

- react-router-dom → para gestionar rutas.
- axios → para realizar peticiones HTTP.
- Dependencias internas de React ya incluidas por create-react-app.

Resultado esperado: El proyecto dispone de todas las librerías necesarias para navegación y comunicación con el backend.

Subtarea 7.3 — Creación del logo y favicon

Descripción: Crear el archivo logo.png de la aplicación y colocarlo en /src/assets/. Configurar el favicon sustituyendo el que viene por defecto en create-react-app.

Resultado esperado: El logo aparece correctamente en el Header y como favicon en el navegador.

Subtarea 7.4 — Implementación del componente Header

Descripción: Crear el componente Header incluyendo:

- Visualización del logo.
- Botones de navegación.
- Comportamiento dinámico según autenticación (Login, Registro, Logout).
- Uso del contexto AuthContext. (se implementa en tareas posteriores)
- Navegación mediante useNavigate.

Resultado esperado: El encabezado aparece en todas las páginas y los botones mostrados cambian según el estado de autenticación.

Subtarea 7.5 — Implementación del componente Footer

Descripción: Crear el componente Footer incluyendo:

- Enlace al GitHub del desarrollador.
- Icono SVG integrado.
- Información de autor y versión.

Resultado esperado: El pie de página aparece en todas las páginas del proyecto.

Subtarea 7.6 — Implementación del componente Home

Descripción: Crear el componente Home mostrando:

- Mensaje de bienvenida.
- Navegación del usuario (UserNav este componente se desarrollará en tareas sucesivas) cuando está autenticado.
- Mensaje alternativo cuando no lo está.

Resultado esperado: La página principal muestra contenido dinámico según el estado del usuario.

Subtarea 7.7 — Creación del RouterApp

Descripción: Crear el componente RouterApp que:

- Incluye Header y Footer para que aparezcan en todas las páginas.
- Define las rutas iniciales:
 - / → Home
 - * → mensaje de página no encontrada

Resultado esperado: La navegación funciona correctamente y los componentes base se renderizan en todas las rutas.

Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 7: Cliente React y componentes base	13 h	02/04/2026	05/04/2026
Subtarea 7.1 — Inicialización proyecto	1 h	02/04/2026	02/04/2026
Subtarea 7.2 — Instalación dependencias	1 h	02/04/2026	02/04/2026
Subtarea 7.3 — Logo y favicon	2 h	02/04/2026	02/04/2026
Subtarea 7.4 — Header	4 h	02/04/2026	03/04/2026
Subtarea 7.5 — Footer	1 h	03/04/2026	03/04/2026
Subtarea 7.6 — Home	2 h	04/04/2026	04/04/2026
Subtarea 7.7 — RouterApp	2 h	04/04/2026	05/04/2026

Tarea 8: Creación de utilidades, componentes de autenticación y servicios del cliente React

Objetivo y metodología

Objetivo. Crear la estructura necesaria para gestionar la autenticación en el cliente React, incluyendo el fichero de constantes, los componentes Registry y Login, el servicio authService, el contexto global AuthContext y el componente RequireAuth para proteger rutas. Estos elementos permiten realizar peticiones al backend, almacenar el token JWT y controlar el acceso a las páginas protegidas.

Metodología. La tarea se desarrolla siguiendo un flujo ordenado:

- **Definición de constantes:** creación del fichero constants.js en /utils para centralizar la URL base de la API y los endpoints utilizados por los servicios.
- **Componentes de autenticación:** implementación de Registry y Login con formularios controlados y llamadas al backend.
- **Servicios de autenticación:** creación de authService.js para gestionar peticiones al backend y almacenamiento del token y del email en localStorage.
- **Contexto global:** implementación de AuthContext para mantener el estado de autenticación en toda la aplicación.
- **Protección de rutas:** creación del componente RequireAuth para restringir el acceso a páginas que requieren autenticación.

Subtareas

Subtarea 8.1 — Creación del fichero de constantes

Descripción: Crear un fichero en /src/utils/constants.js para almacenar:

- La URL base de la API en desarrollo (<http://localhost:8080/api>).
- La URL base cuando se usa Docker (/api) por que crearemos una red interna entre el cliente y el servidor con Docker.
- Los endpoints del backend agrupados por funcionalidad.

Resultado esperado: Todas las rutas del backend quedan centralizadas en un único fichero, facilitando su mantenimiento.

Subtarea 8.2 — Implementación del componente Registry

Descripción: Crear el componente Registry, incluyendo:

- Formulario controlado con email y contraseña.
- Llamada al servicio registerUser.
- Uso del contexto AuthContext para iniciar sesión automáticamente tras registrarse.
- Redirección a /profile tras el registro.

Resultado esperado: El usuario puede registrarse y quedar autenticado automáticamente.

Subtarea 8.3 — Implementación del componente Login

Descripción: Crear el componente Login, incluyendo:

- Formulario controlado con email y contraseña.
- Llamada al servicio loginUser.
- Manejo de errores mostrando mensajes en pantalla.
- Redirección a /profile tras iniciar sesión.

Resultado esperado: El usuario puede iniciar sesión y acceder a las páginas protegidas.

Subtarea 8.4 — Implementación del servicio authService

Descripción: Crear el fichero /src/services/authService.js con las funciones necesarias para:

- Enviar credenciales al backend (loginUser).
- Registrar usuarios (registerUser).
- Guardar y recuperar token y email desde localStorage.
- Eliminar los datos de autenticación al cerrar sesión.

Resultado esperado: El cliente puede comunicarse con el backend y gestionar el token JWT de forma persistente.

Subtarea 8.5 — Implementación del AuthContext

Descripción: Crear el contexto global de autenticación con:

- Estado user que contiene email y token.
- Carga inicial desde localStorage para mantener la sesión tras recargar la página.
- Métodos login y logout.
- Exposición de isAuthenticated y loading.

Resultado esperado: Toda la aplicación puede acceder al estado de autenticación.

Subtarea 8.6 — Implementación del componente RequireAuth

Descripción: Crear el componente RequireAuth para proteger rutas. Incluye:

- Comprobación del estado loading.
- Redirección a /login si el usuario no está autenticado.
- Renderizado del componente protegido si la autenticación es válida.
- Uso del mismo en el enrutador "AppRouter"

Resultado esperado: Las páginas protegidas solo son accesibles cuando el usuario ha iniciado sesión.

Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 8: Utilidades y autenticación en React	24 h	06/04/2026	12/04/2026
Subtarea 8.1 — Fichero de constantes	1 h	06/04/2026	06/04/2026
Subtarea 8.2 — Registry	4 h	06/04/2026	06/04/2026
Subtarea 8.3 — Login	4 h	07/04/2026	07/04/2026
Subtarea 8.4 — authService	5 h	07/04/2026	09/04/2026
Subtarea 8.5 — AuthContext	5 h	09/04/2026	10/04/2026
Subtarea 8.6 — RequireAuth	5 h	11/04/2026	12/04/2026

Tarea 9: Pruebas funcionales del cliente React (autenticación y navegación)

Objetivo y metodología

Objetivo. Verificar el correcto funcionamiento de los componentes de autenticación (Registry, Login), del servicio authService, del contexto global AuthContext, del componente RequireAuth y del comportamiento dinámico del Header. Las pruebas se realizan directamente desde el cliente React, sin Postman, comprobando que las rutas se forman correctamente, que el token se almacena y se utiliza, y que la interfaz responde adecuadamente al estado de autenticación.

Metodología. La validación se realiza mediante pruebas manuales en el navegador, interactuando con los componentes y observando:

- Las peticiones enviadas al backend.
- El contenido de localStorage.
- La navegación entre páginas.
- Los cambios en la interfaz según el estado de autenticación.
- El manejo de errores en pantalla o en consola.

Subtareas

Subtarea 9.1 — Verificación de las rutas generadas por authService

Descripción: Comprobar que las rutas utilizadas por authService se forman correctamente combinando las constantes declaradas en utils/constants:

- API_BASE_URL
- AUTH_ENDPOINTS.LOGIN
- AUTH_ENDPOINTS.REGISTER

Las pruebas consisten en:

- Revisar en la pestaña *Network* del navegador que las peticiones se envían a:
 - `http://localhost:8080/api/auth/register`
 - `http://localhost:8080/api/auth/login`
- Confirmar que no aparecen rutas incorrectas ni duplicadas.

Resultado esperado: Las URLs generadas coinciden exactamente con las definidas en el fichero de constantes y llegan a los controladores del servidor sin problema.

Subtarea 9.2 — Pruebas del componente Registry

Descripción: Probar el comportamiento del componente Registry:

- Introducir email y contraseña válidos.
- Enviar el formulario.
- Verificar que:
 - Se realiza la petición `POST /auth/register`.
 - Se recibe un token en la respuesta.
 - El token y email se almacena, en `localStorage`.
 - El contexto `AuthContext` actualiza el estado del usuario.
 - El sistema redirige automáticamente a `/profile`.
- Probar un registro inválido (email ya existente o error de red).
 - Debe mostrarse un error en la consola.

Resultado esperado: El registro funciona correctamente y el usuario queda autenticado. En caso de error, se muestra en consola.

Subtarea 9.3 — Pruebas del componente Login

Descripción: Probar el comportamiento del componente Login:

- Introducir credenciales válidas.

- Verificar que:
 - Se realiza la petición POST /auth/login.
 - Se recibe un token.
 - El token y el email se guardan en localStorage.
 - El contexto actualiza el estado del usuario.
 - Se redirige a /profile.
- Introducir credenciales incorrectas.
 - El componente debe mostrar el mensaje de error en pantalla.
 - No debe redirigir.
 - No debe guardar nada en localStorage.

Resultado esperado: El login funciona correctamente y los errores se muestran en pantalla.

Subtarea 9.4 — Verificación del comportamiento del Header

Descripción: Comprobar que el Header responde al estado de autenticación:

- **Sin token:**
 - Se muestran los botones: *Inicio*, *Login*, *Registro*.
 - No aparece el email del usuario.
 - No aparece el botón *Logout*.
- **Con token:**
 - Se muestra el email del usuario.
 - Aparece el botón *Logout*.
 - Desaparecen los botones *Login* y *Registro*.
 - El botón *Logout* elimina token y email y devuelve al estado no autenticado.

Resultado esperado: El encabezado cambia dinámicamente según isAuthenticated.

Subtarea 9.5 — Pruebas del AuthContext

Descripción: Validar el comportamiento del contexto global:

- Tras login o registro:
 - user contiene { email, token }.
 - isAuthenticated es true.
- Tras recargar la página:
 - El contexto debe reconstruir el estado desde localStorage.
 - El usuario debe permanecer autenticado.
- Tras logout:
 - Se eliminan token y email de localStorage.
 - user pasa a null.
 - isAuthenticated es false.

Resultado esperado: El estado de autenticación se mantiene correctamente entre recargas y se limpia al cerrar sesión.

Subtarea 9.6 — Pruebas del componente RequireAuth

Descripción: Probar el comportamiento del componente que protege rutas:

- Acceder a una ruta protegida sin estar autenticado.
 - Debe redirigir automáticamente a /login.
- Acceder a una ruta protegida estando autenticado.
 - Debe mostrar el contenido protegido.
- Recargar la página en una ruta protegida.
 - Si hay token en localStorage, debe permitir el acceso.
 - Si no lo hay, debe redirigir a /login.

Resultado esperado: Las rutas protegidas solo son accesibles con autenticación válida.

Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 9: Pruebas funcionales del cliente React	10 h	13/04/2026	14/04/2026
Subtarea 9.1 — Verificación de rutas	1 h	13/04/2026	13/04/2026
Subtarea 9.2 — Pruebas Registry	2 h	13/04/2026	13/04/2026
Subtarea 9.3 — Pruebas Login	2 h	13/04/2026	13/04/2026
Subtarea 9.4 — Pruebas Header	2 h	14/04/2026	14/04/2026
Subtarea 9.5 — Pruebas AuthContext	2 h	14/04/2026	14/04/2026
Subtarea 9.6 — Pruebas RequireAuth	1 h	14/04/2026	14/04/2026

Tarea 10: Creación y pruebas del componente Profile y del servicio necesario userService

Objetivo y metodología

Objetivo. Implementar el componente Profile y el servicio userService, encargados de obtener y mostrar el estado financiero del usuario autenticado. Verificar que las peticiones al backend se realizan correctamente, que el token se utiliza de forma adecuada, que los datos se muestran en pantalla y que la interfaz responde correctamente a los distintos estados (carga, éxito, error).

Metodología. La validación se realiza mediante pruebas funcionales desde el cliente React en el navegador, comprobando:

- La correcta formación de la URL de la petición.
- El uso del token JWT almacenado en localStorage.
- La carga de datos mediante getUserState().
- La actualización del estado interno del componente.
- La visualización del resumen financiero y de las últimas transacciones.
- El comportamiento del componente ante errores de comunicación.

Subtareas

Subtarea 10.1 — Implementación del servicio userService

Descripción: Crear el servicio userService.js con la función getUserState(), que:

- Obtiene el token mediante getToken().
- Construye la URL usando API_BASE_URL + USER_ENDPOINTS.STATE.
- Realiza una petición GET con cabecera Authorization: Bearer <token>.
- Devuelve los datos del usuario autenticado en formato JSON.

Resultado esperado: El servicio devuelve correctamente el estado financiero del usuario cuando el token es válido.

Subtarea 10.2 — Implementación del componente Profile

Descripción: Crear el componente Profile, incluyendo:

- Obtención del usuario desde AuthContext.
- Llamada a getUserState() dentro de useEffect.
- Almacenamiento de los datos en profileData.
- Renderizado del resumen financiero mediante FinancialSummary.
- Visualización de las últimas transacciones (pastTransactions).
- Manejo de errores mediante console.error.

Resultado esperado: El componente muestra correctamente la información del usuario y su estado financiero.

Subtarea 10.3 — Pruebas del servicio userService

Descripción: Validar el comportamiento de getUserState():

- Comprobar en *Network* que la URL generada es:
http://localhost:8080/api/user/state (o /api/user/state en Docker).
- Verificar que la cabecera Authorization incluye el token almacenado.
- Confirmar que la respuesta contiene los campos esperados:
 - currentBalance
 - totalIncomeThisMonth
 - totalExpenseThisMonth
 - dailyAvailable
 - pastTransactions
 - futureTransactions
- Probar el caso sin token:
 - Debe lanzar el error "No token found".

Resultado esperado: El servicio forma correctamente la URL, envía el token y devuelve los datos esperados.

Subtarea 10.4 — Pruebas del componente Profile

Descripción: Validar el comportamiento del componente:

1. Carga correcta de datos

- Estar autenticado.
- Acceder a /profile.
- Verificar que:
 - Se ejecuta `getUserState()`.
 - `profileData` se actualiza.
 - Se muestra el email del usuario.
 - Se muestra el resumen financiero con valores formateados.
 - Se muestran hasta 5 transacciones recientes.

2. Manejo de errores

- Forzar un error (por ejemplo, detener el backend).
- Verificar que:
 - Se muestra un error en consola.
 - La interfaz no se rompe.
 - `FinancialSummary` no se renderiza si `profileData` es null.

3. Formateo de moneda

- Confirmar que `formatCurrency()` muestra valores en formato EUR con dos decimales.

Resultado esperado: El componente responde correctamente tanto en escenarios válidos como en casos de error.

Subtarea 10.5 — Verificación de interacción con AuthContext y navegación

Descripción: Comprobar que:

- El componente utiliza correctamente user.email desde el contexto.
- Si el usuario no está autenticado, RequireAuth (tarea anterior) evita acceder a /profile.
- Tras login o registro, la redirección a /profile permite cargar el componente sin errores.

Resultado esperado: El componente funciona correctamente dentro del flujo de autenticación existente.

Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 10: Componente Profile y userService	12 h	15/04/2026	16/04/2026
Subtarea 10.1 — userService	3 h	15/04/2026	15/04/2026
Subtarea 10.2 — Profile	3 h	15/04/2026	15/04/2026
Subtarea 10.3 — Pruebas userService	2 h	16/04/2026	16/04/2026
Subtarea 10.4 — Pruebas Profile	3 h	16/04/2026	16/04/2026
Subtarea 10.5 — Verificación con AuthContext	1 h	16/04/2026	16/04/2026

Tarea 11: Creación y pruebas del componente Category y del servicio categoryService

Objetivo y metodología

Objetivo. Implementar el componente Category y el servicio categoryService, encargados de obtener y crear categorías asociadas al usuario autenticado. Verificar que las peticiones al backend se realizan correctamente, que el token se utiliza de forma adecuada, que las categorías se cargan y se muestran en pantalla, y que la creación de nuevas categorías actualiza el estado del componente.

Metodología. Las pruebas se realizan directamente desde el cliente React, comprobando:

- La correcta formación de las rutas utilizadas por el servicio.
- El uso del token JWT en las cabeceras.
- La carga inicial de categorías mediante getCategory().
- La creación de nuevas categorías mediante createCategory().
- La actualización del estado interno del componente.
- El comportamiento ante errores de comunicación.

Subtareas

Subtarea 11.1 — Implementación del servicio categoryService

Descripción: Crear el servicio categoryService con dos funciones:

- **getCategory()**
 - Obtiene el token.
 - Construye la URL con API_BASE_URL + CATEGORY_ENDPOINTS.ALL.
 - Realiza una petición GET con cabecera Authorization: Bearer <token>.
 - Devuelve la lista de categorías del usuario.

- **createCategory(name)**
 - Obtiene el token.
 - Construye la URL con API_BASE_URL + CATEGORY_ENDPOINTS.CREATE.
 - Realiza una petición POST con el nombre de la categoría.
 - Devuelve la categoría creada.

Resultado esperado: El servicio permite obtener y crear categorías correctamente.

Subtarea 11.2 — Implementación del componente Category

Descripción: Crear el componente Category con:

- Estado categories para almacenar la lista.
- Estado newCategory para el formulario.
- Carga inicial de categorías mediante useEffect.
- Formulario para crear nuevas categorías.
- Actualización del listado tras crear una categoría.
- Manejo de errores mediante console.error.
- Visualización de las categorías en una lista.

Resultado esperado: El usuario puede ver sus categorías y crear nuevas desde la interfaz.

Subtarea 11.3 — Pruebas del servicio categoryService

Descripción: Validar el comportamiento del servicio:

1. Verificación de rutas

- Confirmar que las peticiones se envían a:
 - GET /categories
 - POST /categories

2. Verificación del token

- Comprobar que la cabecera Authorization incluye el token almacenado.

3. Respuesta correcta

- Confirmar que getCategory() devuelve un array de categorías.
- Confirmar que createCategory() devuelve un objeto con id y name.

4. Manejo de errores

- Forzar un error (backend apagado o token inválido).
- Verificar que se lanza un error y se muestra en consola.

Resultado esperado: El servicio funciona correctamente y responde de forma coherente ante errores.

Subtarea 11.4 — Pruebas del componente Category

Descripción: Validar el comportamiento del componente:

1. Carga inicial

- Estar autenticado.
- Acceder a /categories.
- Verificar que:
 - Se ejecuta getCategory().
 - El estado categories se actualiza.
 - Se muestran todas las categorías en la lista.

2. Creación de categorías

- Introducir un nombre válido.
- Enviar el formulario.
- Verificar que:
 - Se ejecuta createCategory().
 - La categoría creada se añade al estado.
 - El campo de texto se limpia.

3. Validación del formulario

- Intentar enviar una categoría vacía.
- Verificar que no se envía la petición.

4. Manejo de errores

- Forzar un error en la creación.
- Verificar que se muestra en consola.

Resultado esperado: El componente responde correctamente en todos los escenarios.

Subtarea 11.5 — Verificación de interacción con AuthContext y navegación

Descripción: Comprobar que:

- El componente solo funciona si el usuario está autenticado (gracias a RequireAuth).
- El token se utiliza correctamente en todas las peticiones.

Resultado esperado: El componente se integra correctamente en el flujo general de la aplicación.

Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 11: Categorías y categoryService	10 h	17/04/2026	18/04/2026
Subtarea 11.1 — categoryService	2 h	17/04/2026	17/04/2026
Subtarea 11.2 — Category	3 h	17/04/2026	17/04/2026
Subtarea 11.3 — Pruebas categoryService	2 h	18/04/2026	18/04/2026
Subtarea 11.4 — Pruebas Category	2 h	18/04/2026	18/04/2026
Subtarea 11.5 — Verificación con AuthContext	1 h	18/04/2026	18/04/2026

Tarea 12: Creación y pruebas del componente Transaction y del servicio transactionService

Objetivo y metodología

Objetivo. Implementar el componente Transaction y el servicio transactionService, responsables de gestionar las transacciones del usuario autenticado: obtenerlas, crearlas y eliminarlas. Verificar que las peticiones al backend funcionan correctamente, que el token se utiliza de forma adecuada, que los datos se muestran en pantalla y que la interfaz responde correctamente a los distintos estados (carga, éxito, error).

Metodología. Las pruebas se realizan directamente desde el cliente React, comprobando:

- La correcta formación de las rutas utilizadas por el servicio.
- El uso del token JWT en todas las peticiones.
- La carga inicial de transacciones y categorías.
- La creación de nuevas transacciones con validación previa.
- La eliminación de transacciones existentes.
- La actualización del estado interno del componente.
- El comportamiento ante errores de comunicación.

Subtareas

Subtarea 12.1 — Implementación del servicio transactionService

Descripción: Crear el servicio transactionService con tres funciones:

1. getTransactions()

- Obtiene el token.
- Construye la URL con API_BASE_URL + TRANSACTION_ENDPOINTS.ALL.
- Realiza una petición GET con cabecera Authorization: Bearer <token>.
- Devuelve la lista de transacciones del usuario.

2. createTransaction(payload)

- Obtiene el token.
- Construye la URL con API_BASE_URL + TRANSACTION_ENDPOINTS.CREATE.
- Envía un POST con el payload que contiene:
 - amount
 - date
 - description
 - type
 - categoryId
 - recurrence
- Devuelve la transacción creada.

3. deleteTransaction(id)

- Obtiene el token.
- Construye la URL con API_BASE_URL + TRANSACTION_ENDPOINTS.DELETE + "/" + id.
- Envía un DELETE.
- No devuelve contenido.

Resultado esperado: El servicio permite obtener, crear y eliminar transacciones correctamente.

Subtarea 12.2 — Implementación del componente Transaction

Descripción: Crear el componente Transaction con:

- Estado transactions para almacenar la lista.
- Estado categories para mostrar opciones en el formulario.
- Estado form para gestionar los campos de creación.
- Estado loading, saving y error para controlar la interfaz.
- Carga inicial de transacciones y categorías mediante Promise.all().

- Formulario con validación básica:
 - Cantidad válida
 - Fecha obligatoria
 - Categoría obligatoria
- Creación de transacciones con actualización inmediata del listado.
- Eliminación de transacciones con confirmación.
- Manejo de errores mediante mensajes en pantalla y consola.
- Listado de transacciones con formato visual diferenciado para ingresos y gastos.

Resultado esperado: El usuario puede ver, crear y eliminar transacciones desde la interfaz.

Subtarea 12.3 — Pruebas del servicio transactionService

Descripción: Validar el comportamiento del servicio:

1. Verificación de rutas

- Confirmar que las peticiones se envían a:
 - GET /transactions
 - POST /transactions
 - DELETE /transactions/{id}

2. Verificación del token

- Comprobar que la cabecera Authorization incluye el token almacenado.

3. Respuesta correcta

- Confirmar que getTransactions() devuelve un array de transacciones.
- Confirmar que createTransaction() devuelve un objeto con los campos esperados.
- Confirmar que deleteTransaction() elimina la transacción sin errores.

4. Manejo de errores

- Forzar un error (backend apagado, token inválido, payload incorrecto).
- Verificar que se lanza un error y se muestra en consola.

Resultado esperado: El servicio funciona correctamente y responde de forma coherente ante errores.

Subtarea 12.4 — Pruebas del componente Transaction

Descripción: Validar el comportamiento del componente:

1. Carga inicial

- Estar autenticado.
- Acceder a /transactions.
- Verificar que:
 - Se ejecutan getTransactions() y getCategory().
 - El estado transactions se actualiza.
 - El estado categories se actualiza.
 - Se muestra el listado o un mensaje si está vacío.

2. Creación de transacciones

- Rellenar el formulario con valores válidos.
- Enviar el formulario.
- Verificar que:
 - Se ejecuta createTransaction().
 - La transacción creada se añade al inicio del listado.
 - El formulario se resetea.
 - No aparece ningún error.

3. Validación del formulario

- Intentar crear una transacción sin cantidad, sin fecha o sin categoría.
- Verificar que se muestra el mensaje de error correspondiente.
- Confirmar que no se envía la petición.

4. Eliminación de transacciones

- Pulsar el botón “Eliminar”.
- Confirmar la ventana emergente.
- Verificar que:
 - Se ejecuta `deleteTransaction(id)`.
 - La transacción desaparece del listado.

5. Manejo de errores

- Forzar un error en la creación o eliminación.
- Verificar que se muestra un mensaje de error en pantalla y en consola.

Resultado esperado: El componente responde correctamente en todos los escenarios.

Subtarea 12.5 — Verificación de interacción con AuthContext y navegación

Descripción: Comprobar que:

- El componente solo funciona si el usuario está autenticado (gracias a `RequireAuth`).
- El token se utiliza correctamente en todas las peticiones.

Resultado esperado: El componente se integra correctamente en el flujo general de la aplicación.

Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 12: Transacciones y <code>transactionService</code>	14 h	19/04/2026	21/04/2026
Subtarea 12.1 — <code>transactionService</code>	3 h	19/04/2026	19/04/2026
Subtarea 12.2 — Transaction	4 h	19/04/2026	20/04/2026
Subtarea 12.3 — Pruebas <code>transactionService</code>	3 h	20/04/2026	20/04/2026
Subtarea 12.4 — Pruebas Transaction	3 h	21/04/2026	21/04/2026
Subtarea 12.5 — Verificación con AuthContext	1 h	21/04/2026	21/04/2026

Tarea 13: Creación del componente Statistics, su servicio asociado y refactorización (extracción de subcomponentes)

Objetivo y metodología

Objetivo. Implementar la pantalla de estadísticas del usuario, que incluye gráficos de líneas, gráficos circulares y listados de transacciones pasadas y futuras. Esta es la pantalla más completa del proyecto, ya que combina datos del backend, cálculos adicionales en el frontend y visualización mediante Chart.js. Además, una vez verificado el correcto funcionamiento inicial, se extraen las partes internas en componentes independientes (PieCard, LineCard, TransactionList) para mejorar la organización del código y evitar duplicación.

Metodología. La tarea se desarrolla en varias fases:

- Instalación de las dependencias necesarias para gráficos.
- Implementación del componente Statistics y del servicio statisticsService.
- Verificación del funcionamiento completo de la pantalla.
- Extracción de subcomponentes reutilizables.
- Repetición de las pruebas para asegurar que la extracción no rompe el comportamiento.

Subtareas

Subtarea 13.1 — Instalación de dependencias para gráficos

Descripción: Instalar las librerías necesarias para renderizar gráficos:

- chart.js
- react-chartjs-2

Estas dependencias permiten mostrar gráficos de líneas y gráficos circulares dentro de la pantalla de estadísticas.

Resultado esperado: Las librerías quedan instaladas y disponibles para su uso en los componentes.

Subtarea 13.2 — Implementación del servicio `statisticsService`

Descripción: Crear el servicio encargado de procesar los datos obtenidos del backend.

Incluye:

- `fetchStatistics()` → obtiene el estado financiero del usuario.
- `groupByCategory()` → agrupa transacciones por categoría y suma sus importes.
- `buildBalanceSeries()` → genera la serie temporal para el gráfico de línea.
- `getPastTransactions()` → ordena transacciones pasadas por fecha descendente.
- `getFutureTransactions()` → ordena transacciones futuras por fecha ascendente.
- `colorFromCategory()` → genera un color consistente para cada categoría.

Resultado esperado: El servicio devuelve datos listos para ser usados por los gráficos y listados.

Subtarea 13.3 — Implementación del componente `Statistics`

Descripción: Crear el componente principal de estadísticas con:

- Estado `stats` y `loading`.
- Carga inicial mediante `fetchStatistics()`.
- Cálculo de:
 - Gastos pasados por categoría
 - Gastos futuros por categoría
 - Ingresos pasados por categoría
 - Ingresos futuros por categoría
 - Serie con la evolución temporal del balance
- Preparación de datos para gráficos circulares y de línea.
- Renderizado de:
 - Gráfico de línea con evolución del saldo
 - Cuatro gráficos circulares (gastos e ingresos por categoría, tanto pasados como futuros de este mes)

- Listado de transacciones pasadas
- Listado de transacciones futuras

Resultado esperado: La pantalla muestra correctamente todas las estadísticas del usuario.

Subtarea 13.4 — Extracción de subcomponentes (refactorización)

Descripción: Extraer las partes internas del componente en componentes independientes:

- PieCard → tarjeta con gráfico circular y total.
- LineCard → tarjeta con gráfico de línea y resumen de saldos.
- TransactionList → listado de transacciones dentro de una tarjeta.

El objetivo es:

- Reducir la complejidad del componente principal.
- Evitar duplicación de código.
- Facilitar el mantenimiento y la reutilización.

Resultado esperado: Los componentes quedan separados y el código de Statistics se simplifica.

Subtarea 13.5 — Pruebas del servicio statisticsService

Descripción: Validar el comportamiento del servicio:

1. fetchStatistics()

- Confirmar que obtiene los datos del backend usando el token.
- Verificar que devuelve todos los campos esperados.

2. groupByCategory()

- Comprobar que agrupa correctamente por categoría.
- Verificar que suma los importes según el tipo indicado.

3. buildBalanceSeries()

- Confirmar que ordena las transacciones por fecha.
- Verificar que calcula el balance acumulado correctamente.

4. `getPastTransactions()` / `getFutureTransactions()`

- Confirmar que ordenan correctamente según la fecha.

5. `colorFromCategory()`

- Verificar que la misma categoría siempre genera el mismo color.

Resultado esperado: El servicio funciona correctamente en todos los casos.

Subtarea 13.6 — Pruebas del componente **Statistics**

Descripción: Validar el comportamiento del componente completo:

1. Carga inicial

- Estar autenticado.
- Acceder a `/statistics`.
- Verificar que:
 - Se muestra el mensaje de carga.
 - Se ejecuta `fetchStatistics()`.
 - Se actualiza el estado `stats`.
 - Se renderizan todos los gráficos y listados.

2. Gráficos circulares

- Confirmar que cada gráfico muestra:
 - Categorías correctas
 - Valores correctos
 - Colores consistentes

3. Gráfico de línea

- Verificar que la serie temporal coincide con los cálculos del servicio.

4. Listados de transacciones

- Confirmar que:
 - Las transacciones pasadas están ordenadas de más reciente a más antigua.
 - Las futuras están ordenadas de más próxima a más lejana.

5. Manejo de errores

- Forzar un error en la carga.
- Verificar que se muestra el mensaje de error.

Resultado esperado: La pantalla funciona correctamente en todos los escenarios.

Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 13: Pantalla de estadísticas y subcomponentes	19 h	22/04/2026	24/04/2026
Subtarea 13.1 — Instalación dependencias	1 h	22/04/2026	22/04/2026
Subtarea 13.2 — statisticsService	3 h	22/04/2026	22/04/2026
Subtarea 13.3 — Statistics	6 h	23/04/2026	23/04/2026
Subtarea 13.4 — Extracción de subcomponentes	4 h	24/04/2026	24/04/2026
Subtarea 13.5 — Pruebas statisticsService	2 h	25/04/2026	25/04/2026
Subtarea 13.6 — Pruebas Statistics	3 h	25/04/2026	25/04/2026

Tarea 14: Creación del componente UserNav, inserción en pantallas protegidas y ajustes finales de estilos

Objetivo y metodología

Objetivo. Crear el componente UserNav para proporcionar una navegación interna entre las páginas protegidas de la aplicación, e insertarlo en todos los componentes que solo pueden ser accedidos por un usuario autenticado. Además, realizar los ajustes finales del archivo styles.css para asegurar una apariencia coherente, agradable y funcional en todas las pantallas. Finalmente, ejecutar pruebas generales del cliente para validar el comportamiento global de la aplicación.

Metodología. La tarea se desarrolla en tres fases:

- Implementación del componente UserNav.
- Inserción del componente en todas las pantallas protegidas.
- Revisión completa de estilos y comportamiento visual.
- Pruebas generales del cliente tras finalizar el desarrollo.

Subtareas

Subtarea 14.1 — Creación del componente UserNav

Descripción: Crear el componente UserNav que contiene los botones de navegación hacia las páginas protegidas:

- Perfil
- Gestión de categorías
- Gestión de transacciones
- Estadísticas

El componente utiliza un <nav> con la clase page-subnav, que debe quedar definida en styles.css.

Resultado esperado: El componente queda disponible para ser utilizado en todas las pantallas protegidas.

Subtarea 14.2 — Inserción de UserNav en pantallas protegidas

Descripción: Añadir UserNav en los siguientes componentes:

- **Home:** Esta pantalla es accesible sin login, pero el UserNav solo se debe mostrar cuando el usuario está autenticado.
- **Category**
- **Profile**
- **Statistics**
- **Transaction**

El objetivo es que el usuario autenticado pueda navegar entre todas las secciones internas sin volver al menú principal.

Resultado esperado: Todas las pantallas protegidas muestran la barra de navegación interna. Y la pantalla Home solo la muestra (mas un saludo personalizado) cuando el usuario está autenticado.

Subtarea 14.3 — Revisión y ajustes del archivo styles.css

Descripción: Revisar el archivo styles.css completo y realizar los ajustes necesarios para:

- Alinear correctamente UserNav en todas las pantallas.
- Garantizar que los botones mantienen un estilo coherente.
- Ajustar márgenes, paddings y espaciados.
- Verificar que los contenedores mantienen la estructura visual esperada.
- Comprobar que los estilos de listas, formularios, tarjetas y gráficos funcionan correctamente.
- Revisar comportamiento responsive en pantallas pequeñas.

Resultado esperado: La aplicación presenta un diseño uniforme, limpio y funcional en todas las pantallas.

Subtarea 14.4 — Pruebas generales del cliente

Descripción: Realizar pruebas completas de navegación y funcionamiento:

1. Navegación interna

- Comprobar que UserNav permite moverse entre todas las páginas protegidas.
- Verificar que los botones funcionan correctamente.

2. Comportamiento según autenticación

- Sin token → UserNav no debe aparecer.
- Con token → UserNav aparece en todas las pantallas protegidas.

3. Estilos

- Revisar que no existan saltos visuales entre pantallas.
- Confirmar que los estilos de botones, tarjetas, formularios y gráficos son coherentes.

4. Flujo completo

- Login → navegación interna → logout.
- Registro → redirección → navegación interna.
- Acceso directo a rutas protegidas → redirección correcta mediante RequireAuth.
- Logout

Resultado esperado: La aplicación funciona correctamente de principio a fin y presenta un diseño estable y coherente.

Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 14: UserNav, estilos y pruebas finales	12 h	26/04/2026	27/04/2026
Subtarea 14.1 — UserNav	2 h	26/04/2026	26/04/2026
Subtarea 14.2 — Inserción en pantallas protegidas	3 h	26/04/2026	26/04/2026
Subtarea 14.3 — Ajustes de estilos	4 h	27/04/2026	27/04/2026
Subtarea 14.4 — Pruebas generales	3 h	27/04/2026	28/04/2026

Tarea 15: Dockerización completa de la aplicación

Objetivo y metodología

Objetivo. Empaquetar y ejecutar toda la aplicación (backend, frontend y base de datos PostgreSQL) mediante Docker y Docker Compose, asegurando que los servicios se comunican correctamente entre sí, que el cliente puede acceder al servidor a través de Nginx y que los datos se persisten en la base de datos incluso tras reinicios completos.

Metodología. La tarea se estructura en pasos incrementales: creación de las imágenes individuales (backend y frontend), configuración de Nginx como proxy para el cliente, ajuste de rutas y CORS, definición del docker-compose.yml con la red interna y volúmenes de datos, y pruebas completas del flujo cliente–servidor–bbdd, incluyendo reinicios para validar la persistencia.

Subtareas

Subtarea 15.1 — Dockerfile del servidor y pruebas de conexión a la BBDD

Descripción: Crear el Dockerfile del backend con dos etapas (build con Maven y ejecución con JDK 21), generar la imagen (pfm-backend) y comprobar:

- Que el contenedor arranca correctamente.
- Que el servidor se conecta sin problemas a la base de datos PostgreSQL (usando las configuraciones `SPRING.DATASOURCE ... url, username, password, driver-class-name`).
- Que el cliente arrancado en local (modo desarrollo) puede seguir accediendo al backend en Docker.

Resultado esperado: Imagen del backend funcional, con conexión correcta a la base de datos.

Subtarea 15.2 — Configuración de Nginx para la red interna

Descripción: Crear el fichero nginx.conf para el frontend con:

- Servir los archivos estáticos desde /usr/share/nginx/html.
- Redirigir todas las rutas de React a index.html (try_files).
- Definir el proxy para /api/ apuntando al servicio backend:8080.
- Mantener las cabeceras necesarias (Host, X-Real-IP, X-Forwarded-*, Origin).

Resultado esperado: Nginx preparado para servir el cliente y reenviar las peticiones /api al backend dentro de la red de Docker.

Subtarea 15.3 — Dockerfile del cliente con Nginx y pruebas de build

Descripción: Crear el Dockerfile del frontend con dos etapas:

- Build con Node (npm ci + npm run build).
- Servir el resultado con Nginx (nginx:stable-alpine) usando el nginx.conf creado.

Probar:

- Que la imagen pfm-frontend se construye correctamente.
- Que se puede acceder al cliente desde http://localhost en el navegador.

Resultado esperado: Imagen del frontend funcional, sirviendo la SPA (Single Page Application) y lista para integrarse con el backend.

Subtarea 15.4 — Ajuste de rutas en el cliente y CORS en el servidor

Descripción:

- Ajustar las constantes del cliente (API_BASE_URL) para que en modo Docker apunten a /api.
- Mantener la configuración de desarrollo (http://localhost:8080/api) para uso sin Docker.
- Ajustar SecurityConfig del servidor para que CORS permita accesos tanto desde el cliente en local como desde el contenedor (orígenes http://localhost, etc.).

Resultado esperado: El cliente puede comunicarse con el backend tanto en entorno local como en entorno Docker sin bloqueos de CORS.

Subtarea 15.5 — Creación del docker-compose.yml

Descripción: Definir el docker-compose.yml final con los servicios:

- **postgres**
 - Imagen oficial.
 - Variables de entorno (POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB).
 - Volumen pgdata para persistencia.
 - healthcheck para asegurar que está listo antes del backend.
- **backend**
 - Imagen pfm-backend:latest.
 - Dependencia de postgres con condición service_healthy. (requiere añadir la dependencia actuator de spring en el servidor y exponer los endpoints (health, info))
 - Variables SPRING_DATASOURCE_URL, SPRING_DATASOURCE_USERNAME, SPRING_DATASOURCE_PASSWORD.
 - Exposición del puerto 8080.
- **frontend**
 - Build desde el proyecto React.
 - Imagen pfm-frontend:latest.
 - Exposición del puerto 80.
 - Dependencia del backend.

Resultado esperado: Archivo docker-compose.yml listo para levantar toda la aplicación con un solo comando. “docker compose up --build”

Subtarea 15.6 — Pruebas completas con docker-compose

Descripción:

- Ejecutar docker compose up --build.
- Verificar que:
 - Los tres servicios (postgres, backend, frontend) se construyen y arrancan sin errores.
 - Se accede al cliente desde http://localhost.
 - Todo el flujo cliente–servidor funciona: registro, login, categorías, transacciones, estadísticas.
 - Los datos se persisten en la base de datos incluso tras parar y volver a levantar los contenedores (incluyendo el volumen de PostgreSQL).

Resultado esperado: La aplicación completa funciona en Docker, con persistencia de datos y flujo cliente–servidor correcto.

Planificación

Tarea / Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 15: Dockerización completa de la aplicación	18 h	28/04/2026	30/04/2026
Subtarea 15.1 — Dockerfile servidor	4 h	28/04/2026	28/04/2026
Subtarea 15.2 — Configuración Nginx	2 h	28/04/2026	29/04/2026
Subtarea 15.3 — Dockerfile cliente y build	3 h	29/04/2026	29/04/2026
Subtarea 15.4 — Ajuste rutas y CORS	3 h	30/04/2026	30/04/2026
Subtarea 15.5 — docker-compose.yml	3 h	30/04/2026	30/04/2026
Subtarea 15.6 — Pruebas completas con docker-compose	3 h	01/05/2026	01/05/2026

Recursos humanos

El desarrollo del proyecto ha sido realizado íntegramente por una única persona. El perfil asignado ha sido:

- **Abraham Quintana Micó — (Analista–desarrollador Full-Stack)** Responsable de todas las fases del proyecto: análisis, diseño, implementación del backend y frontend, integración, despliegue con Docker, pruebas y documentación.

No se han requerido perfiles adicionales, ya que el alcance del proyecto y la planificación permitían su ejecución completa por un único desarrollador.

Recursos materiales

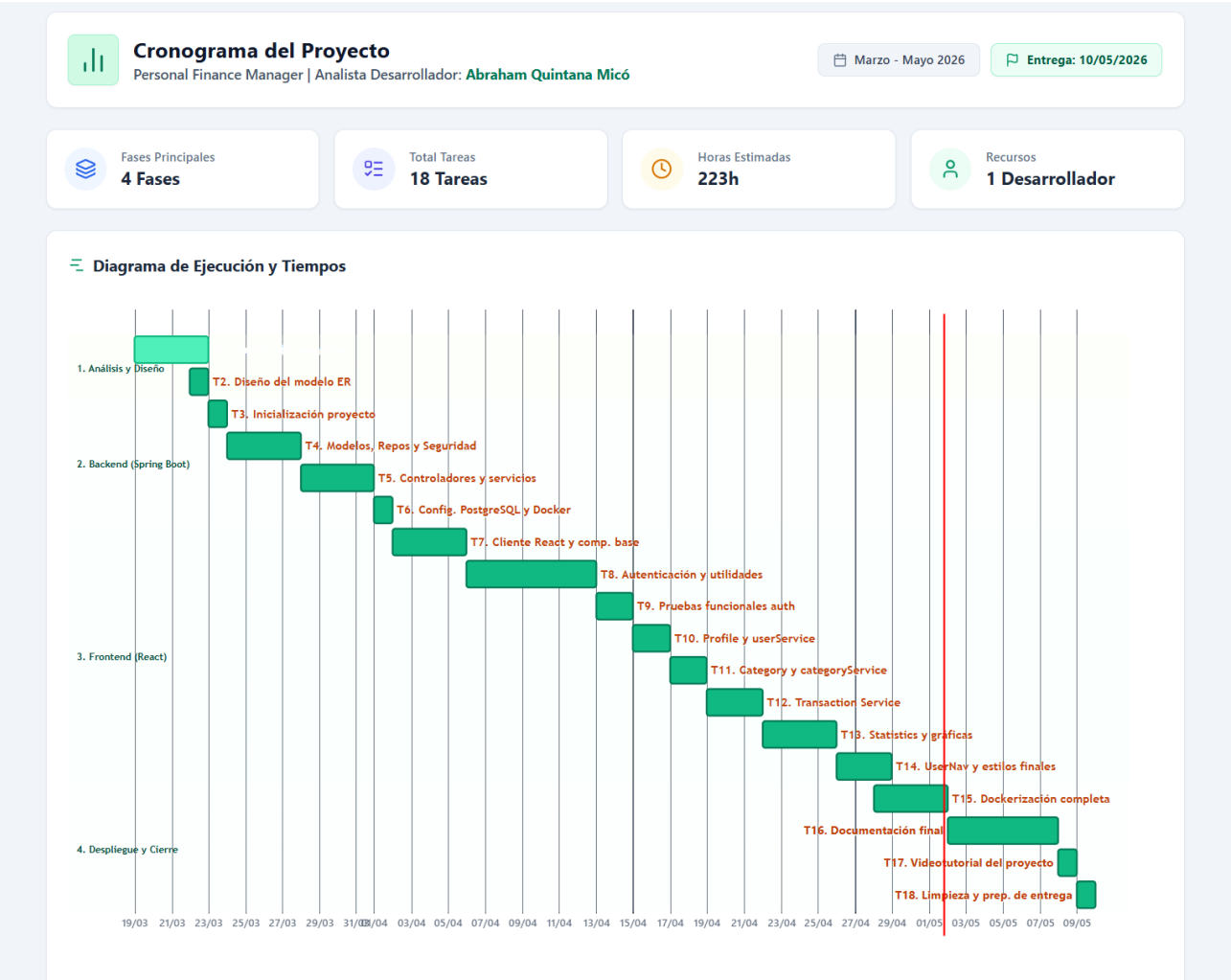
Para llevar a cabo el proyecto se ha utilizado un entorno de trabajo de escritorio con las siguientes características mínimas:

- Ordenador con **32 GB de RAM**,
- Unidad **SSD M.2 de al menos 500 GB**,
- Procesador **Intel i5** de gama media-alta,
- **Doble monitor**, para facilitar el desarrollo simultáneo a las pruebas y visualización de los cambios del backend, frontend y documentación.

Además, se han utilizado herramientas de software habituales en el desarrollo web moderno:

- Sistema operativo de escritorio,
- VisualStudioCode,
- IntelliJ
- Navegador actualizado,
- Docker, DockerDesktop y Docker Compose,
- Git para control de versiones,
- Dbeaver para acceder a bbdd y generar diagramas,
- Terminal y Powershell para la ejecución de comandos (arranques y paradas de los componentes, instalaciones, accesos a bbdd, ...)

CRONOGRAMA



PRESUPUESTO

Tarea / Subtarea	Horas	Recursos humanos (€)	Recursos materiales (€)	Total (€)
Tarea 1: Análisis de requisitos	10.0	350,00	0,00	350,00
Subtarea 1.1 - Investigación tecnologías	4.0	140,00	0,00	140,00
Subtarea 1.2 - Requisitos funcionales/no funcionales	4.0	140,00	0,00	140,00
Subtarea 1.3 - Diseño alcance	2.0	70,00	0,00	70,00
Tarea 2: Diseño ER	4.5	157,50	0,00	157,50
Subtarea 2.1 - Análisis técnico	1.0	35,00	0,00	35,00
Subtarea 2.2 - Diseño conceptual	1.0	35,00	0,00	35,00
Subtarea 2.3 - Enums y reglas	0.5	17,50	0,00	17,50
Subtarea 2.4 - Normalización	1.0	35,00	0,00	35,00
Subtarea 2.5 - Diagramación	1.0	35,00	0,00	35,00
Tarea 3: Inicialización proyecto	4.5	157,50	0,00	157,50
Subtarea 3.1 - Spring Initializr	1.0	35,00	0,00	35,00
Subtarea 3.2 - Revisión pom.xml	0.5	17,50	0,00	17,50
Subtarea 3.3 - Estructura paquetes	1.5	52,50	0,00	52,50
Subtarea 3.4 - Verificación build	1.5	52,50	0,00	52,50
Tarea 4: Implementación modelos y seguridad	20.0	700,00	0,00	700,00
Subtarea 4.1 - Modelos y repositorios	6.0	210,00	0,00	210,00
Subtarea 4.2 - DTOs y mappers	4.0	140,00	0,00	140,00
Subtarea 4.3 - Configuración seguridad y JWT	4.0	140,00	0,00	140,00
Subtarea 4.4 - AuthController y AuthService	3.0	105,00	0,00	105,00
Subtarea 4.5 - Pruebas Postman y manejo errores	3.0	105,00	0,00	105,00
Tarea 5: Controladores y servicios	23.0	805,00	0,00	805,00
Subtarea 5.1 - UserController	3.0	105,00	0,00	105,00
Subtarea 5.2 - CategoryController	4.0	140,00	0,00	140,00
Subtarea 5.3 - TransactionController	4.0	140,00	0,00	140,00
Subtarea 5.4 - AuthService	2.0	70,00	0,00	70,00
Subtarea 5.5 - UserFinancialStateService	8.0	105,00	0,00	105,00
Subtarea 5.6 - Pruebas Postman	2.0	70,00	0,00	70,00

Tarea 6: PostgreSQL con Docker	8.0	280,00	0,00	280,00
Subtarea 6.1 - Docker Compose Postgres	2.0	70,00	0,00	70,00
Subtarea 6.2 - Modificar application.properties	2.0	70,00	0,00	70,00
Subtarea 6.3 - Verificación persistencia/relaciones	2.0	70,00	0,00	70,00
Subtarea 6.4 - Verificación encriptación contraseñas	2.0	70,00	0,00	70,00
Tarea 7: Cliente React y componentes base	10.0	350,00	0,00	350,00
Subtarea 7.1 - Inicialización CRA	1.0	35,00	0,00	35,00
Subtarea 7.2 - Instalación dependencias	1.0	35,00	0,00	35,00
Subtarea 7.3 - Logo y favicon	1.0	35,00	0,00	35,00
Subtarea 7.4 - Header	2.0	70,00	0,00	70,00
Subtarea 7.5 - Footer	1.0	35,00	0,00	35,00
Subtarea 7.6 - Home	2.0	70,00	0,00	70,00
Subtarea 7.7 - RouterApp	2.0	70,00	0,00	70,00
Tarea 8: Utilidades y autenticación (actualizada)	24.0	840,00	0,00	840,00
Subtarea 8.1 - Fichero constantes	1.0	35,00	0,00	35,00
Subtarea 8.2 - Registry	4.0	140,00	0,00	140,00
Subtarea 8.3 - Login	4.0	140,00	0,00	140,00
Subtarea 8.4 - authService	5.0	175,00	0,00	175,00
Subtarea 8.5 - AuthContext	5.0	175,00	0,00	175,00
Subtarea 8.6 - RequireAuth	5.0	175,00	0,00	175,00
Tarea 9: Pruebas funcionales cliente (auth & nav)	10.0	350,00	0,00	350,00
Subtarea 9.1 - Verificación rutas	1.0	35,00	0,00	35,00
Subtarea 9.2 - Pruebas Registry	2.0	70,00	0,00	70,00
Subtarea 9.3 - Pruebas Login	2.0	70,00	0,00	70,00
Subtarea 9.4 - Pruebas Header	2.0	70,00	0,00	70,00
Subtarea 9.5 - Pruebas AuthContext	2.0	70,00	0,00	70,00
Subtarea 9.6 - Pruebas RequireAuth	1.0	35,00	0,00	35,00
Tarea 10: Profile & userService	12.0	420,00	0,00	420,00
Subtarea 10.1 - userService	3.0	105,00	0,00	105,00
Subtarea 10.2 - Profile	3.0	105,00	0,00	105,00
Subtarea 10.3 - Pruebas userService	2.0	70,00	0,00	70,00

Subtarea 10.4 - Pruebas Profile	3.0	105,00	0,00	105,00
Subtarea 10.5 - Verificación AuthContext	1.0	35,00	0,00	35,00
Tarea 11: Category & categoryService	10.0	350,00	0,00	350,00
Subtarea 11.1 - categoryService	2.0	70,00	0,00	70,00
Subtarea 11.2 - Category	3.0	105,00	0,00	105,00
Subtarea 11.3 - Pruebas categoryService	2.0	70,00	0,00	70,00
Subtarea 11.4 - Pruebas Category	2.0	70,00	0,00	70,00
Subtarea 11.5 - Verificación AuthContext	1.0	35,00	0,00	35,00
Tarea 12: Transaction & transactionService	14.0	490,00	0,00	490,00
Subtarea 12.1 - transactionService	3.0	105,00	0,00	105,00
Subtarea 12.2 - Transaction	4.0	140,00	0,00	140,00
Subtarea 12.3 - Pruebas transactionService	3.0	105,00	0,00	105,00
Subtarea 12.4 - Pruebas Transaction	3.0	105,00	0,00	105,00
Subtarea 12.5 - Verificación AuthContext	1.0	35,00	0,00	35,00
Tarea 13: Statistics & subcomponentes	20.0	700,00	0,00	700,00
Subtarea 13.1 - Instalación dependencias	1.0	35,00	0,00	35,00
Subtarea 13.2 - statisticsService	3.0	105,00	0,00	105,00
Subtarea 13.3 - Statistics	6.0	210,00	0,00	210,00
Subtarea 13.4 - Extracción subcomponentes	4.0	140,00	0,00	140,00
Subtarea 13.5 - Pruebas statisticsService	2.0	70,00	0,00	70,00
Subtarea 13.6 - Pruebas Statistics	3.0	105,00	0,00	105,00
Subtarea 13.7 - Pruebas tras refactorización	1.0	35,00	0,00	35,00
Tarea 14: UserNav, estilos y pruebas finales	12.0	420,00	0,00	420,00
Subtarea 14.1 - UserNav	2.0	70,00	0,00	70,00
Subtarea 14.2 - Inserción pantallas protegidas	3.0	105,00	0,00	105,00
Subtarea 14.3 - Ajustes styles.css	4.0	140,00	0,00	140,00
Subtarea 14.4 - Pruebas generales	3.0	105,00	0,00	105,00
Tarea 15: Dockerización completa	18.0	630,00	0,00	630,00
Subtarea 15.1 - Dockerfile servidor	4.0	140,00	0,00	140,00
Subtarea 15.2 - Nginx	2.0	70,00	0,00	70,00
Subtarea 15.3 - Dockerfile cliente	3.0	105,00	0,00	105,00

Subtarea 15.4 - Ajuste rutas y CORS	3.0	105,00	0,00	105,00
Subtarea 15.5 - docker-compose.yml	3.0	105,00	0,00	105,00
Subtarea 15.6 - Pruebas docker-compose	3.0	105,00	0,00	105,00
Tarea 16: Documentación final	18.0	630,00	0,00	630,00
Documentación del proyecto (memoria, anexos, entrega)	18.0	630,00	0,00	630,00
Tarea 17: Videotutorial	2.0	70,00	0,00	70,00
Tarea 18: Limpieza y preparación de entrega	3.0	105,00	0,00	105,00
Resumen materiales (PC + pantallas)	—	—	1.500,00	1.500,00
TOTAL (recursos humanos)	—	7.805,00		
TOTAL (recursos materiales)	—	—	1.500,00	
SUMA TOTALES (humanos + materiales)	—	7.805,00	1.500,00	9.305,00

Política de seguimiento y evaluación

Durante todo el desarrollo he aplicado una serie de prácticas que me han permitido mantener el control del proyecto, detectar desviaciones y asegurar que cada fase cumplía los objetivos previstos. Las acciones principales han sido las siguientes:

- **Planificación estructurada del trabajo.** Antes de comenzar cada bloque definí claramente las tareas y subtareas, junto con sus fechas de inicio y finalización. Esto permitió avanzar con un orden lógico y evitar acumulación de trabajo.
- **Revisión continua del progreso.** Al finalizar cada jornada revisé el estado de las tareas completadas, comprobando que se ajustaban a los requisitos funcionales y técnicos establecidos. Esta autoevaluación diaria ayudó a corregir errores de forma temprana.
- **Control temporal mediante planificación detallada.** El desglose de tareas y subtareas permitió llevar un seguimiento preciso del avance, comparando el progreso real con el planificado y ajustando el ritmo cuando fue necesario.
- **Uso de Git como herramienta de control de versiones.** El versionado continuo facilitó mantener un historial claro de cambios, revertir errores cuando fue necesario y organizar el desarrollo en fases coherentes.
- **Pruebas manuales frecuentes.** Cada funcionalidad fue probada inmediatamente después de implementarse. Esto incluyó flujos críticos como autenticación, gestión de categorías, transacciones, cálculos financieros y estadísticas.
- **Validación final tras la dockerización.** Una vez integrados backend, frontend y base de datos en Docker, se realizaron pruebas completas del flujo de la aplicación para asegurar que todo funcionaba correctamente en un entorno equivalente al de despliegue.

Gracias a estas medidas, el proyecto se mantuvo dentro de los plazos previstos y con un nivel de calidad estable durante todo su desarrollo.

Futuras mejoras

A continuación se enumeran las mejoras previstas para versiones posteriores de la aplicación:

- Avisos de futuros gastos (seguros, hipotecas, tarjetas, suscripciones...).
- Avisos cuando se exceda la media diaria disponible.
- Cálculo y visualización de intereses mensuales de deudas.
- Confirmación manual de ingresos (si son recurrentes), antes de incorporarlos al balance.
- Gestión de crédito disponible (tarjetas).
- Registro de disponibilidad económica para emergencias (amigos y familiares).
- Gestión de ahorros (huchas y cuentas de ahorro).
 - Integración con el balance mensual para cubrir déficits automáticamente.
 - Transferencia automática del sobrante mensual a los ahorros.
- Registro y seguimiento de inversiones en activos.
- Mejoras de UX y mensajes de error más claros en el cliente.
- Sistema de roles y panel de administración:
 - Gestión de usuarios,
 - Reinicio de contraseñas,
 - Estadísticas de uso y métricas del sistema.
- Validación avanzada de contraseñas en el cliente, forzando el uso de contraseñas complejas.
- Eliminación de categorías con advertencias sobre transacciones recurrentes asociadas o sistema de desactivación. (no se ha implementado porque requiere lógica de eliminación en cascada y control de seguridad de la misma)

Anexos

Junto con este documento se incluyen los recursos complementarios del proyecto:

- **Presentación para la defensa del proyecto**, en formato LibreOffice Impress.
- **Video-tutorial**, con explicación del funcionamiento y demostración de la aplicación.
- **Repositorio completo del proyecto**, disponible en GitHub:
<https://github.com/AbrahamQM/personal-finance-manager>

El repositorio contiene:

- Un **README.md general** en la raíz, con instrucciones de uso y despliegue.
- Un **README.md específico** en la carpeta del backend.
- Un **README.md específico** en la carpeta del frontend.
- La **licencia** correspondiente.
- Todo el Javadoc generado en formato navegable desde navegador web en la carpeta:
/personal-finance-manager/tree/main/personal-finance-manager/personal-finance-manager/Javadocs
- Fichero .gitignore para evitar agregar al repositorio información sensible o no necesaria como son por ejemplo los contenido en la carpeta Target del servidor o la carpeta node_modules del cliente.
- Fichero docker-compose.yml con la configuración necesaria para levantar todo el proyecto.
- **Código fuente completo**, organizado en backend, frontend y configuración Docker.
- **Ficheros necesarios para la ejecución local y en Docker**, incluyendo docker-compose.yml, Dockerfile del servidor, Dockerfile del cliente y nginx.conf.

Bibliografía

- SpringBoot Initializr para la composición inicial del servidor [SpringBoot Initializr](#)
- Guías y tutoriales de Baeldung para Spring <https://www.baeldung.com/>
- Documentación oficial de React <https://react.dev/>
- Documentación de Chart.js <https://www.chartjs.org/docs/latest/>
- Documentación oficial de Docker <https://docs.docker.com/>
- Repositorio de oficial de imágenes de docker <https://hub.docker.com/>
- Documentación oficial de PostgreSQL <https://www.postgresql.org/docs/>